ПРИЛОЖЕНИЕ А

Текст веб-приложения

АННОТАЦИЯ

Данное приложение содержит исходный текст информационной системы агрохолдинга «Колос-Юг»: серверной части на PHP, клиентской части на React и скрипта создания базы данных MySQL.

ТЕКСТ ПРОГРАММЫ

Текст файла api/cancelOrder.php

```
<?php
require __DIR__ . '/db.php';
requireMethod('POST');
$user = requireAuth();

$id = (int)(input()['id'] ?? 0);
$pdo->beginTransaction();
$stmt = $pdo->prepare('SELECT id_crop, id_warehouse, quantity_t, status FROM orders
WHERE id_order = ? AND id_user = ? FOR UPDATE');
$stmt->execute([$id, $user['id_user']]);
$o = $stmt->fetch();
if (!$o) {
    $pdo->rollBack();
    respond(['message' => 'Заказ не найден'], 404);
}
if (!in_array($o['status'], ['new', 'confirmed'], true)) {
    $pdo->rollBack();
    respond(['message' => 'Отгруженный или завершённый заказ отменить нельзя'], 422);
}
$pdo->prepare("UPDATE orders SET status = 'cancelled' WHERE id_order = ?")-
>execute([$id]);
$pdo->prepare('UPDATE stock SET quantity_t = quantity_t + ? WHERE id_crop = ? AND
id_warehouse = ?')
    ->execute([$o['quantity_t'], $o['id_crop'], $o['id_warehouse']]);
$pdo->commit();
respond(['message' => 'Заказ отменён, продукция возвращена на склад']);
```

Текст файла api/confirmReceipt.php

```
<?php
require __DIR__ . '/db.php';
requireMethod('POST');
$user = requireAuth();

$id = (int)(input()['id'] ?? 0);
$stmt = $pdo->prepare("UPDATE orders SET status = 'completed' WHERE id_order = ? AND
id_user = ? AND status = 'shipped'");
$stmt->execute([$id, $user['id_user']]);
if ($stmt->rowCount() === 0) {
    respond(['message' => 'Подтвердить получение можно только для отгруженного
заказа'], 422);
}
respond(['message' => 'Получение подтверждено, заказ завершён']);
```

Текст файла api/crop.php

```
<?php
require __DIR__ . '/db.php';

$id = (int)($_GET['id'] ?? 0);
if ($id <= 0) {
    respond(['message' => 'Некорректный идентификатор продукции'], 400);
}
$stmt = $pdo->prepare('SELECT id_crop, name, category, grade, description,
price_per_ton, image FROM crops WHERE id_crop = ? AND is_active = 1');
$stmt->execute([$id]);
$crop = $stmt->fetch();
if (!$crop) {
    respond(['message' => 'Продукция не найдена'], 404);
}

$stmt = $pdo->prepare(
    'SELECT w.id_warehouse, w.name, w.type, w.address, f.region, s.quantity_t
     FROM stock s JOIN warehouses w ON w.id_warehouse = s.id_warehouse JOIN farms f ON
f.id_farm = w.id_farm
     WHERE s.id_crop = ? ORDER BY s.quantity_t DESC'
);
$stmt->execute([$id]);
$crop['stock'] = $stmt->fetchAll();

$stmt = $pdo->prepare(
    'SELECT s.season, SUM(s.area_ha) AS area_ha, COALESCE(SUM(h.amount_t), 0) AS
amount_t
     FROM sowings s LEFT JOIN harvests h ON h.id_sowing = s.id_sowing
     WHERE s.id_crop = ? GROUP BY s.season ORDER BY s.season DESC'
);
$stmt->execute([$id]);
$crop['seasons'] = $stmt->fetchAll();

respond(['crop' => $crop]);
```

Текст файла api/crops.php

```
<?php
require __DIR__ . '/db.php';

$where = ['c.is_active = 1'];
$params = [];
if (!empty($_GET['category'])) {
    $where[] = 'c.category = ?';
    $params[] = clean($_GET['category'], 30);
}
if (!empty($_GET['q'])) {
    $where[] = '(c.name LIKE ? OR c.grade LIKE ?)';
    $like = '%' . clean($_GET['q'], 60) . '%';
    array_push($params, $like, $like);
}
if (!empty($_GET['priceMax'])) {
    $where[] = 'c.price_per_ton <= ?';
    $params[] = (float)$_GET['priceMax'];
}
$having = !empty($_GET['inStock']) ? 'HAVING stock_t > 0' : '';
$order = match ($_GET['sort'] ?? '') {
    'price_desc' => 'c.price_per_ton DESC',
    'stock' => 'stock_t DESC',
    'name' => 'c.name ASC',
    default => 'c.price_per_ton ASC',
};
```

```
$stmt = $pdo->prepare(
    'SELECT c.id_crop, c.name, c.category, c.grade, c.price_per_ton, c.image,
            COALESCE(SUM(s.quantity_t), 0) AS stock_t,
            COUNT(DISTINCT CASE WHEN s.quantity_t > 0 THEN s.id_warehouse END) AS
warehouses
     FROM crops c LEFT JOIN stock s ON s.id_crop = c.id_crop
     WHERE ' . implode(' AND ', $where) . '
     GROUP BY c.id_crop ' . $having . ' ORDER BY ' . $order
);
$stmt->execute($params);
respond(['crops' => $stmt->fetchAll()]);
```

Текст файла api/db.php

```
<?php
declare(strict_types=1);


ini_set('display_errors', '0');
header('Content-Type: application/json; charset=utf-8');
header('X-Content-Type-Options: nosniff');


session_set_cookie_params(['httponly' => true, 'samesite' => 'Lax', 'path' => '/']);
session_start();


const DB_HOST = 'localhost';
const DB_NAME = 'agro_db';
const DB_USER = 'root';
const DB_PASS = '';


try {
    $pdo = new PDO('mysql:host=' . DB_HOST . ';dbname=' . DB_NAME . ';charset=utf8mb4',
DB_USER, DB_PASS, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]);
} catch (PDOException $e) {
    respond(['message' => 'Нет подключения к базе данных'], 500);
}


function respond(array $data, int $code = 200): never
{
    http_response_code($code);
    echo json_encode($data, JSON_UNESCAPED_UNICODE);
    exit;
}


function input(): array
{
    $data = json_decode(file_get_contents('php://input') ?: '', true);
    return is_array($data) ? $data : [];
}


function requireMethod(string $method): void
{
    if ($_SERVER['REQUEST_METHOD'] !== $method) {
        respond(['message' => 'Метод не поддерживается'], 405);
    }
}


function currentUser(): ?array
{
```

```
    return $_SESSION['user'] ?? null;
}

function requireAuth(array $roles = []): array
{
    $user = currentUser();
    if (!$user) {
        respond(['message' => 'Требуется авторизация'], 401);
    }
    if ($roles && !in_array($user['role'], $roles, true)) {
        respond(['message' => 'Недостаточно прав'], 403);
    }
    return $user;
}

function clean(mixed $value, int $max = 255): string
{
    return mb_substr(trim(strip_tags((string)($value ?? ''))), 0, $max);
}

function validPhone(string $phone): bool
{
    return (bool)preg_match('/^\+?[0-9\s\-\(\)]{10,20}$/', $phone);
}
```

Текст файла api/farms.php

```
<?php
require __DIR__ . '/db.php';

$rows = $pdo->query(
    "SELECT f.id_farm, f.name, f.region, f.district, f.area_ha, f.specialization,
f.image,
            (SELECT COUNT(*) FROM fields fl WHERE fl.id_farm = f.id_farm) AS
fields_count,
            (SELECT COALESCE(SUM(h.amount_t), 0) FROM harvests h JOIN sowings s ON
s.id_sowing = h.id_sowing
               JOIN fields fl ON fl.id_field = s.id_field WHERE fl.id_farm = f.id_farm
AND s.season = 2026) AS harvest_t,
            (SELECT COALESCE(SUM(w.capacity_t), 0) FROM warehouses w WHERE w.id_farm =
f.id_farm) AS capacity_t
     FROM farms f ORDER BY f.area_ha DESC"
)->fetchAll();
respond(['farms' => $rows]);
```

Текст файла api/login.php

```
<?php
require __DIR__ . '/db.php';
requireMethod('POST');

$d = input();
$email = mb_strtolower(clean($d['email'] ?? '', 120));
$password = (string)($d['password'] ?? '');

if ($email === '' || $password === '') {
    respond(['message' => 'Заполните все поля'], 422);
}

$attempts = $_SESSION['login_attempts'] ?? ['count' => 0, 'time' => time()];
if ($attempts['count'] >= 5 && time() - $attempts['time'] < 300) {
    respond(['message' => 'Слишком много попыток. Повторите через 5 минут'], 429);
}
```

```
$stmt = $pdo->prepare('SELECT id_user, last_name, first_name, company, inn, email,
phone, password_hash, role FROM users WHERE email = ?');
$stmt->execute([$email]);
$user = $stmt->fetch();


if (!$user || !password_verify($password, $user['password_hash'])) {
    $_SESSION['login_attempts'] = ['count' => $attempts['count'] + 1, 'time' =>
time()];
    respond(['message' => 'Неверный e-mail или пароль'], 401);
}


unset($user['password_hash'], $_SESSION['login_attempts']);
session_regenerate_id(true);
$_SESSION['user'] = $user;
respond(['user' => $user]);
```

Текст файла api/logout.php

```
<?php
require __DIR__ . '/db.php';
$_SESSION = [];
session_destroy();
respond(['message' => 'Вы вышли из аккаунта']);
```

Текст файла api/me.php

```
<?php
require __DIR__ . '/db.php';
respond(['user' => currentUser()]);
```

Текст файла api/myOrders.php

```
<?php
require __DIR__ . '/db.php';
$user = requireAuth();


$stmt = $pdo->prepare(
    'SELECT o.id_order, o.quantity_t, o.price_per_ton, o.total_price, o.delivery,
o.address, o.status, o.created_at,
            c.id_crop, c.name AS crop, c.image, w.name AS warehouse
     FROM orders o
     JOIN crops c ON c.id_crop = o.id_crop
     JOIN warehouses w ON w.id_warehouse = o.id_warehouse
     WHERE o.id_user = ? ORDER BY o.created_at DESC'
);
$stmt->execute([$user['id_user']]);
respond(['orders' => $stmt->fetchAll(), 'user' => $user]);
```

Текст файла api/order.php

```
<?php
require __DIR__ . '/db.php';
requireMethod('POST');
$user = requireAuth(['client']);


$d = input();
$cropId = (int)($d['cropId'] ?? 0);
$warehouseId = (int)($d['warehouseId'] ?? 0);
$qty = round((float)($d['quantity'] ?? 0), 1);
$delivery = in_array($d['delivery'] ?? '', ['Самовывоз', 'Автодоставка', 'Железная
дорога'], true) ? $d['delivery'] : null;
$address = clean($d['address'] ?? '', 200);
$comment = clean($d['comment'] ?? '', 500);


if ($cropId <= 0 || $warehouseId <= 0 || !$delivery) {
```

```
    respond(['message' => 'Выберите склад отгрузки и способ доставки'], 422);
}
if ($qty < 5 || $qty > 5000) {
    respond(['message' => 'Объём заказа должен быть от 5 до 5 000 тонн'], 422);
}
if ($delivery !== 'Самовывоз' && mb_strlen($address) < 10) {
    respond(['message' => 'Укажите адрес доставки'], 422);
}


try {
    $pdo->beginTransaction();
    $stmt = $pdo->prepare(
        'SELECT s.id_stock, s.quantity_t, c.price_per_ton FROM stock s JOIN crops c ON
c.id_crop = s.id_crop
         WHERE s.id_crop = ? AND s.id_warehouse = ? AND c.is_active = 1 FOR UPDATE'
    );
    $stmt->execute([$cropId, $warehouseId]);
    $row = $stmt->fetch();
    if (!$row) {
        $pdo->rollBack();
        respond(['message' => 'На выбранном складе нет этой продукции'], 404);
    }
    if ((float)$row['quantity_t'] < $qty) {
        $pdo->rollBack();
        respond(['message' => 'Недостаточно продукции на складе. Доступно: ' .
$row['quantity_t'] . ' т'], 409);
    }
    $total = round($qty * (float)$row['price_per_ton'], 2);
    $pdo->prepare('UPDATE stock SET quantity_t = quantity_t - ? WHERE id_stock = ?')-
>execute([$qty, $row['id_stock']]);
    $pdo->prepare(
        'INSERT INTO orders (id_user, id_crop, id_warehouse, quantity_t, price_per_ton,
total_price, delivery, address, comment)
         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)'
    )->execute([$user['id_user'], $cropId, $warehouseId, $qty, $row['price_per_ton'],
$total, $delivery,
        $delivery === 'Самовывоз' ? null : $address, $comment ?: null]);
    $id = (int)$pdo->lastInsertId();
    $pdo->commit();
} catch (PDOException $e) {
    if ($pdo->inTransaction()) {
        $pdo->rollBack();
    }
    respond(['message' => 'Не удалось оформить заказ'], 500);
}


respond(['message' => 'Заказ № ' . $id . ' оформлен. Менеджер подтвердит его и выставит
счёт', 'id' => $id], 201);
```

Текст файла api/register.php

```
<?php
require __DIR__ . '/db.php';
requireMethod('POST');


$d = input();
$lastName = clean($d['lastName'] ?? '', 60);
$firstName = clean($d['firstName'] ?? '', 60);
$company = clean($d['company'] ?? '', 150);
$inn = clean($d['inn'] ?? '', 12);
$email = mb_strtolower(clean($d['email'] ?? '', 120));
$phone = clean($d['phone'] ?? '', 20);
$password = (string)($d['password'] ?? '');
```

```
if ($lastName === '' || $firstName === '' || $company === '' || $inn === '' || $email
=== '' || $phone === '' || $password === '') {
    respond(['message' => 'Заполните все поля'], 422);
}

$errors = [];
if (!preg_match('/^[A-ЯЁa-яёA-Za-z\- ]{2,60}$/u', $lastName) || !preg_match('/^[A-ЯЁa-
яёA-Za-z\- ]{2,60}$/u', $firstName)) {
    $errors[] = 'Имя и фамилия могут содержать только буквы и дефис';
}
if (!preg_match('/^(\d{10}|\d{12})$/', $inn)) {
    $errors[] = 'ИНН должен состоять из 10 цифр для организации или 12 цифр для ИП';
}
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = 'Некорректный e-mail';
}
if (!validPhone($phone)) {
    $errors[] = 'Некорректный номер телефона';
}
if (mb_strlen($password) < 8 || !preg_match('/\d/', $password) || !preg_match('/[A-Za-
zA-Яа-я]/u', $password)) {
    $errors[] = 'Пароль должен быть не короче 8 символов и содержать буквы и цифры';
}
if ($errors) {
    respond(['message' => implode('. ', $errors)], 422);
}

$stmt = $pdo->prepare('SELECT id_user FROM users WHERE email = ?');
$stmt->execute([$email]);
if ($stmt->fetch()) {
    respond(['message' => 'Пользователь с таким e-mail уже зарегистрирован'], 409);
}

$stmt = $pdo->prepare('INSERT INTO users (last_name, first_name, company, inn, email,
phone, password_hash) VALUES (?, ?, ?, ?, ?, ?, ?)');
$stmt->execute([$lastName, $firstName, $company, $inn, $email, $phone,
password_hash($password, PASSWORD_DEFAULT)]);

respond(['message' => 'Регистрация прошла успешно. Войдите в аккаунт'], 201);
```

Текст файла api/sendRequest.php

```
<?php
require __DIR__ . '/db.php';
requireMethod('POST');

$d = input();
$name = clean($d['name'] ?? '', 100);
$company = clean($d['company'] ?? '', 150);
$phone = clean($d['phone'] ?? '', 20);
$email = clean($d['email'] ?? '', 120);
$topics = ['Закупка продукции', 'Сотрудничество', 'Аренда земли', 'Работа в компании',
'Другое'];
$topic = in_array($d['topic'] ?? '', $topics, true) ? $d['topic'] : 'Закупка
продукции';
$message = clean($d['message'] ?? '', 1000);

if ($name === '' || $phone === '') {
    respond(['message' => 'Укажите имя и телефон'], 422);
}
if (!validPhone($phone)) {
    respond(['message' => 'Некорректный номер телефона'], 422);
}
if ($email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
```

```
    respond(['message' => 'Некорректный e-mail'], 422);
}
if (empty($d['agree'])) {
    respond(['message' => 'Необходимо согласие на обработку персональных данных'],
422);
}

$pdo->prepare('INSERT INTO requests (name, company, phone, email, topic, message)
VALUES (?, ?, ?, ?, ?, ?)')
    ->execute([$name, $company ?: null, $phone, $email ?: null, $topic, $message ?:
null]);
respond(['message' => 'Заявка отправлена. Менеджер свяжется с вами в течение рабочего
дня'], 201);
```

Текст файла api/stats.php

```
<?php
require __DIR__ . '/db.php';

$one = fn(string $sql) => $pdo->query($sql)->fetchColumn();
respond([
    'farms' => (int)$one('SELECT COUNT(*) FROM farms'),
    'area' => (float)$one('SELECT COALESCE(SUM(area_ha), 0) FROM farms'),
    'harvest' => (float)$one('SELECT COALESCE(SUM(h.amount_t), 0) FROM harvests h JOIN
sowings s ON s.id_sowing = h.id_sowing WHERE s.season = 2026'),
    'capacity' => (float)$one('SELECT COALESCE(SUM(capacity_t), 0) FROM warehouses'),
]);
```

Текст файла api/admin/config.php

```
<?php
require __DIR__ . '/../db.php';

const TABLES = [
    'users' => ['pk' => 'id_user', 'title' => 'Пользователи', 'roles' => ['admin'],
'hidden' => ['password_hash']],
    'farms' => ['pk' => 'id_farm', 'title' => 'Хозяйства', 'roles' => ['admin']],
    'fields' => ['pk' => 'id_field', 'title' => 'Поля', 'roles' => ['admin']],
    'crops' => ['pk' => 'id_crop', 'title' => 'Культуры', 'roles' => ['admin']],
    'sowings' => ['pk' => 'id_sowing', 'title' => 'Посевы', 'roles' => ['admin',
'manager']],
    'harvests' => ['pk' => 'id_harvest', 'title' => 'Урожай', 'roles' => ['admin',
'manager']],
    'warehouses' => ['pk' => 'id_warehouse', 'title' => 'Склады', 'roles' =>
['admin']],
    'stock' => ['pk' => 'id_stock', 'title' => 'Остатки', 'roles' => ['admin',
'manager']],
    'orders' => ['pk' => 'id_order', 'title' => 'Заказы', 'roles' => ['admin',
'manager']],
    'requests' => ['pk' => 'id_request', 'title' => 'Заявки', 'roles' => ['admin',
'manager']],
];

const VIEWS = [
    'view1_products_catalog' => 'Каталог продукции с остатками',
    'view2_warehouse_load' => 'Загрузка складов',
    'view3_orders_full' => 'Заказы с данными покупателей',
    'view4_revenue_by_crop' => 'Выручка по культурам',
    'view5_field_yield' => 'Урожайность по полям',
    'view6_farm_areas' => 'Площади хозяйств',
    'view7_plan_vs_fact' => 'План и факт урожая',
    'view8_client_stats' => 'Статистика по покупателям',
    'view9_new_requests' => 'Необработанные заявки',
    'view10_monthly_sales' => 'Продажи по месяцам',
];
```

```
function tableConfig(string $table, array $user): array
{
    if (!isset(TABLES[$table])) {
        respond(['message' => 'Неизвестная таблица'], 400);
    }
    $cfg = TABLES[$table];
    if (!in_array($user['role'], $cfg['roles'], true)) {
        respond(['message' => 'Недостаточно прав для этой таблицы'], 403);
    }
    return $cfg;
}

function tableColumns(PDO $pdo, string $table): array
{
    $stmt = $pdo->prepare(
        'SELECT COLUMN_NAME AS name, DATA_TYPE AS type, COLUMN_TYPE AS column_type,
IS_NULLABLE AS nullable,
                COLUMN_DEFAULT AS def, EXTRA AS extra
         FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?
ORDER BY ORDINAL_POSITION'
    );
    $stmt->execute([DB_NAME, $table]);
    return $stmt->fetchAll();
}

function prepareValues(PDO $pdo, string $table, array $row, bool $isInsert): array
{
    $cfg = TABLES[$table];
    $values = [];
    foreach (tableColumns($pdo, $table) as $col) {
        $name = $col['name'];
        if ($name === $cfg['pk'] || str_contains($col['extra'], 'auto_increment')) {
            continue;
        }
        if ($table === 'users' && $name === 'password_hash') {
            $password = (string)($row['password'] ?? '');
            if ($password !== '') {
                if (mb_strlen($password) < 8) {
                    respond(['message' => 'Пароль должен быть не короче 8 символов'],
422);
                }
                $values[$name] = password_hash($password, PASSWORD_DEFAULT);
            } elseif ($isInsert) {
                respond(['message' => 'Укажите пароль нового пользователя'], 422);
            }
            continue;
        }
        if (!array_key_exists($name, $row)) {
            continue;
        }
        $value = $row[$name];
        if ($value === '' || $value === null) {
            if ($col['nullable'] === 'NO' && $col['def'] === null) {
                respond(['message' => 'Поле «' . $name . '» обязательно для
заполнения'], 422);
            }
            if ($col['nullable'] === 'YES') {
                $values[$name] = null;
            }
            continue;
        }
        $value = trim(strip_tags((string)$value));
        if (in_array($col['type'], ['int', 'tinyint', 'smallint', 'decimal'], true) &&
!is_numeric($value)) {
            respond(['message' => 'Поле «' . $name . '» должно быть числом'], 422);
```

```
        }
        if ($col['type'] === 'date' && !preg_match('/^\d{4}-\d{2}-\d{2}$/', $value)) {
            respond(['message' => 'Поле «' . $name . '» должно быть датой в формате
ГГГГ-ММ-ДД'], 422);
        }
        if ($col['type'] === 'enum') {
            preg_match_all("/'((?:[^']|'')*)'/", $col['column_type'], $m);
            if (!in_array($value, $m[1], true)) {
                respond(['message' => 'Недопустимое значение поля «' . $name . '»'],
422);
            }
        }
        if ($name === 'email' && !filter_var($value, FILTER_VALIDATE_EMAIL)) {
            respond(['message' => 'Некорректный e-mail'], 422);
        }
        $values[$name] = $value;
    }
    if (!$values) {
        respond(['message' => 'Нет данных для сохранения'], 422);
    }
    return $values;
}

function dbError(PDOException $e): never
{
    $code = $e->errorInfo[1] ?? 0;
    $message = match ((int)$code) {
        1062 => 'Запись с таким значением уже существует',
        1451 => 'Нельзя удалить запись: на неё ссылаются другие таблицы',
        1452 => 'Указан несуществующий внешний ключ',
        4025, 3819 => 'Значение нарушает ограничение таблицы',
        default => 'Ошибка базы данных',
    };
    respond(['message' => $message], 422);
}
```

Текст файла api/admin/deleteRow.php

```
<?php
require __DIR__ . '/config.php';
requireMethod('POST');
$user = requireAuth(['admin', 'manager']);

$d = input();
$table = (string)($d['table'] ?? '');
$cfg = tableConfig($table, $user);
$id = (int)($d['id'] ?? 0);

if ($table === 'users' && $id === (int)$user['id_user']) {
    respond(['message' => 'Нельзя удалить собственную учётную запись'], 422);
}

try {
    $stmt = $pdo->prepare("DELETE FROM `$table` WHERE `{$cfg['pk']}` = ?");
    $stmt->execute([$id]);
} catch (PDOException $e) {
    dbError($e);
}
if ($stmt->rowCount() === 0) {
    respond(['message' => 'Запись не найдена'], 404);
}
respond(['message' => 'Запись удалена']);
```

Текст файла api/admin/getTable.php

```
<?php
require __DIR__ . '/config.php';
$user = requireAuth(['admin', 'manager']);

$table = (string)($_GET['table'] ?? '');
$cfg = tableConfig($table, $user);

$columns = array_values(array_filter(
    tableColumns($pdo, $table),
    fn($c) => !in_array($c['name'], $cfg['hidden'] ?? [], true)
));
foreach ($columns as &$c) {
    if ($c['type'] === 'enum') {
        preg_match_all("/'((?:[^']|'')*)'/", $c['column_type'], $m);
        $c['options'] = $m[1];
    }
}
unset($c);

$select = implode(', ', array_map(fn($c) => '`' . $c['name'] . '`', $columns));
$rows = $pdo->query("SELECT $select FROM `$table` ORDER BY `{$cfg['pk']}` DESC")-
>fetchAll();

respond(['table' => $table, 'title' => $cfg['title'], 'pk' => $cfg['pk'], 'columns' =>
$columns, 'rows' => $rows,
    'needsPassword' => $table === 'users']);
```

Текст файла api/admin/getView.php

```
<?php
require __DIR__ . '/config.php';
requireAuth(['admin', 'manager']);

$view = (string)($_GET['view'] ?? '');
if (!isset(VIEWS[$view])) {
    respond(['message' => 'Неизвестное представление'], 400);
}
$rows = $pdo->query("SELECT * FROM `$view`")->fetchAll();
respond(['view' => $view, 'title' => VIEWS[$view], 'rows' => $rows]);
```

Текст файла api/admin/insertRow.php

```
<?php
require __DIR__ . '/config.php';
requireMethod('POST');
$user = requireAuth(['admin', 'manager']);

$d = input();
$table = (string)($d['table'] ?? '');
tableConfig($table, $user);
$values = prepareValues($pdo, $table, (array)($d['row'] ?? []), true);

$cols = implode(', ', array_map(fn($c) => "`$c`", array_keys($values)));
$marks = implode(', ', array_fill(0, count($values), '?'));
try {
    $pdo->prepare("INSERT INTO `$table` ($cols) VALUES ($marks)")-
>execute(array_values($values));
} catch (PDOException $e) {
    dbError($e);
}
respond(['message' => 'Запись добавлена', 'id' => (int)$pdo->lastInsertId()], 201);
```

Текст файла api/admin/meta.php

```
<?php
require __DIR__ . '/config.php';
$user = requireAuth(['admin', 'manager']);

$tables = [];
foreach (TABLES as $name => $cfg) {
    if (in_array($user['role'], $cfg['roles'], true)) {
        $tables[] = ['name' => $name, 'title' => $cfg['title']];
    }
}
$views = [];
foreach (VIEWS as $name => $title) {
    $views[] = ['name' => $name, 'title' => $title];
}
respond(['tables' => $tables, 'views' => $views, 'user' => $user]);
```

Текст файла api/admin/stats.php

```
<?php
require __DIR__ . '/config.php';
requireAuth(['admin', 'manager']);

$one = fn(string $sql) => $pdo->query($sql)->fetchColumn();
respond([
    'newOrders' => (int)$one("SELECT COUNT(*) FROM orders WHERE status = 'new'"),
    'newRequests' => (int)$one("SELECT COUNT(*) FROM requests WHERE status = 'new'"),
    'stock' => (float)$one('SELECT COALESCE(SUM(quantity_t), 0) FROM stock'),
    'revenue' => (float)$one("SELECT COALESCE(SUM(total_price), 0) FROM orders WHERE
status IN ('confirmed', 'shipped', 'completed')"),
]);
```

Текст файла api/admin/updateRow.php

```
<?php
require __DIR__ . '/config.php';
requireMethod('POST');
$user = requireAuth(['admin', 'manager']);

$d = input();
$table = (string)($d['table'] ?? '');
$cfg = tableConfig($table, $user);
$id = (int)($d['id'] ?? 0);
$values = prepareValues($pdo, $table, (array)($d['row'] ?? []), false);

$set = implode(', ', array_map(fn($c) => "`$c` = ?", array_keys($values)));
try {
    $stmt = $pdo->prepare("UPDATE `$table` SET $set WHERE `{$cfg['pk']}` = ?");
    $stmt->execute([...array_values($values), $id]);
} catch (PDOException $e) {
    dbError($e);
}
respond(['message' => 'Изменения сохранены']);
```

Текст файла vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  base: '/agro/',
  plugins: [react()],
  build: { outDir: 'dist', assetsDir: 'assets' },
})
```

Текст файла index.html

```
<!doctype html>
<html lang="ru">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="Агрохолдинг «Колос-Юг»: производство и продажа
зерновых, масличных и бобовых культур оптом с элеваторов юга России." />
    <link rel="icon" type="image/svg+xml" href="/agro/favicon.svg" />
    <title>Колос-Юг - агрохолдинг</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Текст файла public/.htaccess

```
RewriteEngine On
RewriteBase /agro/
RewriteRule ^index\.html$ - [L]
RewriteCond %{REQUEST_URI} !^/agro/api/
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule . /agro/index.html [L]
```

Текст файла src/main.jsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import '@fontsource/lora/600.css'
import '@fontsource/lora/700.css'
import '@fontsource/inter/400.css'
import '@fontsource/inter/500.css'
import '@fontsource/inter/600.css'
import './styles/base.css'
import App from './App.jsx'


ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter basename="/agro">
      <App />
    </BrowserRouter>
  </React.StrictMode>
)
```

Текст файла src/App.jsx

```
import { useEffect } from 'react'
import { Routes, Route, Navigate, useLocation } from 'react-router-dom'
import { AuthProvider, useAuth } from './AuthContext.jsx'
import Header from './components/Header.jsx'
import Footer from './components/Footer.jsx'
import AuthModal from './components/AuthModal.jsx'
import Home from './pages/Home.jsx'
import Products from './pages/Products.jsx'
import Product from './pages/Product.jsx'
import Farms from './pages/Farms.jsx'
import About from './pages/About.jsx'
import Contacts from './pages/Contacts.jsx'
import Account from './pages/Account.jsx'
import Admin from './pages/Admin.jsx'
import NotFound from './pages/NotFound.jsx'


function ScrollTop() {
```

```
  const { pathname } = useLocation()
  useEffect(() => {
    window.scrollTo(0, 0)
  }, [pathname])
  return null
}

function Protected({ roles, children }) {
  const { user, ready, setModal } = useAuth()
  useEffect(() => {
    if (ready && !user) setModal('login')
  }, [ready, user, setModal])
  if (!ready) return <div className="page-loader">Загрузка...</div>
  if (!user) return <Navigate to="/" replace />
  if (roles && !roles.includes(user.role)) return <Navigate to="/account" replace />
  return children
}

export default function App() {
  return (
    <AuthProvider>
      <ScrollTop />
      <Header />
      <main>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/products" element={<Products />} />
          <Route path="/products/:id" element={<Product />} />
          <Route path="/farms" element={<Farms />} />
          <Route path="/about" element={<About />} />
          <Route path="/contacts" element={<Contacts />} />
          <Route path="/account" element={<Protected><Account /></Protected>} />
          <Route path="/admin" element={<Protected roles={['admin', 'manager']}><Admin
/></Protected>} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </main>
      <Footer />
      <AuthModal />
    </AuthProvider>
  )
}
```

Текст файла src/AuthContext.jsx

```
import { createContext, useContext, useEffect, useState } from 'react'
import { get, post } from './api.js'

const AuthContext = createContext(null)

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null)
  const [ready, setReady] = useState(false)
  const [modal, setModal] = useState(null)

  useEffect(() => {
    get('me.php')
      .then((d) => setUser(d.user))
      .catch(() => setUser(null))
      .finally(() => setReady(true))
  }, [])

  const login = async (email, password) => {
    const d = await post('login.php', { email, password })
```

```
    setUser(d.user)
    return d.user
  }

  const logout = async () => {
    await post('logout.php', {})
    setUser(null)
  }

  return (
    <AuthContext.Provider value={{ user, ready, login, logout, modal, setModal }}>
      {children}
    </AuthContext.Provider>
  )
}

export const useAuth = () => useContext(AuthContext)
```

Текст файла src/api.js

```
const API = import.meta.env.BASE_URL + 'api/'

async function request(path, options = {}) {
  const res = await fetch(API + path, {
    credentials: 'same-origin',
    headers: { 'Content-Type': 'application/json' },
    ...options,
  })
  let data = {}
  try {
    data = await res.json()
  } catch {
    data = { message: 'Сервер вернул некорректный ответ' }
  }
  if (!res.ok) {
    const error = new Error(data.message || 'Ошибка запроса')
    error.status = res.status
    throw error
  }
  return data
}

export const get = (path) => request(path)
export const post = (path, body) => request(path, { method: 'POST', body:
JSON.stringify(body) })

export const img = (file) => import.meta.env.BASE_URL + 'img/' + (file ||
'placeholder.jpg')

export const money = (v) =>
  new Intl.NumberFormat('ru-RU', { style: 'currency', currency: 'RUB',
maximumFractionDigits: 0 }).format(Number(v || 0))

export const num = (v, d = 0) => new Intl.NumberFormat('ru-RU', {
maximumFractionDigits: d }).format(Number(v || 0))

export const date = (v) =>
  v ? new Date(v.replace(' ', 'T')).toLocaleDateString('ru-RU', { day: 'numeric',
month: 'long', year: 'numeric' }) : '-'

export const STATUS = {
  new: 'Новый',
```

```
  confirmed: 'Подтверждён',
  shipped: 'Отгружен',
  completed: 'Завершён',
  cancelled: 'Отменён',
}

export const CATEGORIES = ['Зерновые', 'Масличные', 'Бобовые', 'Технические']
```

Текст файла src/components/AuthModal.jsx

```
import { useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { useAuth } from '../AuthContext.jsx'
import { post } from '../api.js'
import '../styles/auth.css'

const emailRe = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/

export default function AuthModal() {
  const { modal, setModal, login } = useAuth()
  const navigate = useNavigate()
  const [form, setForm] = useState({ lastName: '', firstName: '', company: '', inn: '',
email: '', phone: '', password: '', password2: '' })
  const [msg, setMsg] = useState({ type: '', text: '' })
  const [busy, setBusy] = useState(false)

  useEffect(() => {
    setMsg({ type: '', text: '' })
  }, [modal])

  useEffect(() => {
    const onKey = (e) => e.key === 'Escape' && setModal(null)
    window.addEventListener('keydown', onKey)
    return () => window.removeEventListener('keydown', onKey)
  }, [setModal])

  if (!modal) return null
  const isLogin = modal === 'login'
  const set = (k) => (e) => setForm({ ...form, [k]: e.target.value })

  const validate = () => {
    if (isLogin) {
      if (!form.email || !form.password) return 'Заполните все поля'
      if (!emailRe.test(form.email)) return 'Некорректный e-mail'
      return ''
    }
    if (!form.lastName || !form.firstName || !form.company || !form.inn || !form.email
|| !form.phone || !form.password) return 'Заполните все поля'
    if (!/^(\d{10}|\d{12})$/.test(form.inn)) return 'ИНН должен состоять из 10 цифр для
организации или 12 цифр для ИП'
    if (!/^[А-ЯЁа-яёA-Za-z\- ]{2,60}$/.test(form.lastName + form.firstName)) return
'Имя и фамилия могут содержать только буквы и дефис'
    if (!emailRe.test(form.email)) return 'Некорректный e-mail'
    if (!/^\+?[0-9\s\-()]{10,20}$/.test(form.phone)) return 'Некорректный номер
телефона'
    if (form.password.length < 8 || !/\d/.test(form.password) || !/[A-Za-zA-Яа-
я]/.test(form.password)) return 'Пароль должен быть не короче 8 символов и содержать
буквы и цифры'
    if (form.password !== form.password2) return 'Пароли не совпадают'
    return ''
  }

  const submit = async (e) => {
```

```
    e.preventDefault()
    const error = validate()
    if (error) return setMsg({ type: 'error', text: error })
    setBusy(true)
    try {
      if (isLogin) {
        const u = await login(form.email, form.password)
        setModal(null)
        navigate(['admin', 'manager'].includes(u.role) ? '/admin' : '/account')
      } else {
        const d = await post('register.php', form)
        setModal('login')
        setTimeout(() => setMsg({ type: 'success', text: d.message }), 0)
      }
    } catch (err) {
      setMsg({ type: 'error', text: err.message })
    } finally {
      setBusy(false)
    }
  }

  return (
    <div className="modal-backdrop" onMouseDown={(e) => e.target === e.currentTarget &&
setModal(null)}>
      <div className="modal" role="dialog" aria-modal="true">
        <button className="modal-close" onClick={() => setModal(null)} aria-
label="Закрыть">×</button>
        <h2>{isLogin ? 'Вход в кабинет покупателя' : 'Регистрация покупателя'}</h2>
        <form onSubmit={submit} noValidate>
          {!isLogin && (
            <>
              <div className="row2">
                <label>Фамилия<input value={form.lastName} onChange={set('lastName')}
/></label>
                <label>Имя<input value={form.firstName} onChange={set('firstName')}
/></label>
              </div>
              <div className="row2">
                <label>Организация<input value={form.company} onChange={set('company')}
placeholder="ООО «Пример»" /></label>
                <label>ИНН<input value={form.inn} onChange={set('inn')}
inputMode="numeric" maxLength="12" /></label>
              </div>
            </>
          )}
          <label>E-mail<input type="email" value={form.email} onChange={set('email')}
autoComplete="email" /></label>
          {!isLogin && <label>Телефон<input value={form.phone} onChange={set('phone')}
placeholder="+7 (900) 000-00-00" /></label>}
          <label>Пароль<input type="password" value={form.password}
onChange={set('password')} autoComplete={isLogin ? 'current-password' : 'new-password'}
/></label>
          {!isLogin && <label>Повторите пароль<input type="password"
value={form.password2} onChange={set('password2')} /></label>}
          {msg.text && <div className={'alert alert-' + msg.type}>{msg.text}</div>}
          <button className="btn btn-green btn-block" disabled={busy}>{isLogin ?
'Войти' : 'Зарегистрироваться'}</button>
        </form>
        <p className="modal-switch">
          {isLogin ? 'Ещё нет аккаунта? ' : 'Уже есть аккаунт? '}
          <button onClick={() => setModal(isLogin ? 'register' : 'login')}>{isLogin ?
'Регистрация' : 'Войти'}</button>
        </p>
      </div>
    </div>
  )
}
```

Текст файла src/components/CallForm.jsx

```
import { useState } from 'react'
import { post } from '../api.js'
import '../styles/callform.css'


const empty = { name: '', company: '', phone: '', email: '', topic: 'Закупка
продукции', message: '', agree: false }


export default function CallForm({ title = 'Оставьте заявку на закупку', subtitle }) {
  const [form, setForm] = useState(empty)
  const [state, setState] = useState({ type: '', text: '' })
  const [sending, setSending] = useState(false)


  const set = (k) => (e) => setForm({ ...form, [k]: e.target.type === 'checkbox' ?
e.target.checked : e.target.value })


  const submit = async (e) => {
    e.preventDefault()
    if (!form.name.trim() || !form.phone.trim()) {
      setState({ type: 'error', text: 'Укажите имя и телефон' })
      return
    }
    setSending(true)
    try {
      const d = await post('sendRequest.php', form)
      setState({ type: 'success', text: d.message })
      setForm(empty)
    } catch (err) {
      setState({ type: 'error', text: err.message })
    } finally {
      setSending(false)
    }
  }


  return (
    <section className="callform" id="callform">
      <div className="container callform-inner">
        <div className="callform-text">
          <h2>{title}</h2>
          <p>{subtitle || 'Менеджер отдела продаж свяжется с вами в течение рабочего
дня, уточнит объём, базис поставки и подготовит коммерческое предложение.'}</p>
          <ul>
            <li>Партии от 5 тонн</li>
            <li>Отгрузка автотранспортом и по железной дороге</li>
            <li>Протоколы качества на каждую партию</li>
          </ul>
        </div>
        <form className="callform-form" onSubmit={submit} noValidate>
          <div className="row2">
            <label>Имя*<input value={form.name} onChange={set('name')}
placeholder="Иван" /></label>
            <label>Компания<input value={form.company} onChange={set('company')}
placeholder="ООО «Пример»" /></label>
          </div>
          <div className="row2">
            <label>Телефон*<input value={form.phone} onChange={set('phone')}
placeholder="+7 (900) 000-00-00" /></label>
            <label>E-mail<input value={form.email} onChange={set('email')}
placeholder="mail@example.ru" /></label>
          </div>
          <label>Тема
            <select value={form.topic} onChange={set('topic')}>
              <option>Закупка продукции</option>
```

```
              <option>Сотрудничество</option>
              <option>Аренда земли</option>
              <option>Работа в компании</option>
              <option>Другое</option>
            </select>
          </label>
          <label>Комментарий<textarea rows="3" value={form.message}
onChange={set('message')} placeholder="Культура, объём, базис поставки, сроки"
/></label>
          <label className="check"><input type="checkbox" checked={form.agree}
onChange={set('agree')} /> Согласен на обработку персональных данных</label>
          {state.text && <div className={'alert alert-' +
state.type}>{state.text}</div>}
          <button className="btn btn-gold" disabled={sending}>{sending ? 'Отправляем…'
: 'Отправить заявку'}</button>
        </form>
      </div>
    </section>
  )
}
```

Текст файла src/components/CropCard.jsx

```
import { Link } from 'react-router-dom'
import { img, money, num } from '../api.js'

export default function CropCard({ crop }) {
  const inStock = Number(crop.stock_t) > 0
  return (
    <article className="crop-card">
      <Link to={'/products/' + crop.id_crop} className="crop-card-img">
        <img src={img(crop.image)} alt={crop.name} loading="lazy" />
        <span className="chip">{crop.category}</span>
      </Link>
      <div className="crop-card-body">
        <h3><Link to={'/products/' + crop.id_crop}>{crop.name}</Link></h3>
        <p className="muted">{crop.grade}</p>
        <dl className="crop-card-facts">
          <div><dt>Цена</dt><dd>{money(crop.price_per_ton)} / т</dd></div>
          <div><dt>В наличии</dt><dd className={inStock ? 'ok' : 'no'}>{inStock ?
num(crop.stock_t) + ' т' : 'нет'}</dd></div>
          <div><dt>Складов</dt><dd>{crop.warehouses}</dd></div>
        </dl>
        <Link to={'/products/' + crop.id_crop} className="btn btn-gold btn-block btn-
sm">Оформить заказ</Link>
      </div>
    </article>
  )
}
```

Текст файла src/components/Footer.jsx

```
import { Link } from 'react-router-dom'
import Logo from './Logo.jsx'
import '../styles/footer.css'

export default function Footer() {
  return (
    <footer className="footer">
      <div className="container footer-grid">
        <div>
          <Logo light />
          <p className="footer-about">Агрохолдинг полного цикла: 6 хозяйств в южных и
центральных регионах России, собственные элеваторы и логистика.</p>
        </div>
        <div className="footer-col">
```

```
          <h4>Покупателям</h4>
          <Link to="/products">Продукция</Link>
          <Link to="/farms">Хозяйства</Link>
          <Link to="/contacts">Контакты</Link>
        </div>
        <div className="footer-col">
          <h4>Связь</h4>
          <a href="tel:+78612001000">+7 (861) 200-10-00</a>
          <a href="mailto:sales@kolos-yug.ru">sales@kolos-yug.ru</a>
          <span>Краснодар, ул. Красная, 120</span>
        </div>
        <div className="footer-col">
          <h4>Реквизиты</h4>
          <span>АО «Колос-Юг»</span>
          <span>ИНН 2310123456</span>
        </div>
      </div>
      <div className="container footer-bottom">© 2026 АО «Колос-Юг». Учебный
проект.</div>
    </footer>
  )
}
```

Текст файла src/components/Header.jsx

```
import { useState } from 'react'
import { Link, NavLink } from 'react-router-dom'
import { useAuth } from '../AuthContext.jsx'
import Logo from './Logo.jsx'
import '../styles/header.css'


export default function Header() {
  const { user, setModal } = useAuth()
  const [open, setOpen] = useState(false)
  const close = () => setOpen(false)


  return (
    <header className="header">
      <div className="topbar">
        <div className="container topbar-inner">
          <span>Производство и оптовая продажа сельхозпродукции с 2004 года</span>
          <a href="tel:+78612001000">+7 (861) 200-10-00</a>
        </div>
      </div>
      <div className="container header-inner">
        <Link to="/" onClick={close} aria-label="На главную"><Logo /></Link>
        <nav className={'nav' + (open ? ' nav-open' : '')}>
          <NavLink to="/products" onClick={close}>Продукция</NavLink>
          <NavLink to="/farms" onClick={close}>Хозяйства</NavLink>
          <NavLink to="/about" onClick={close}>О холдинге</NavLink>
          <NavLink to="/contacts" onClick={close}>Контакты</NavLink>
          {user && ['admin', 'manager'].includes(user.role) && <NavLink to="/admin"
onClick={close}>Управление</NavLink>}
        </nav>
        <div className="header-actions">
          {user ? (
            <Link to="/account" className="btn btn-line btn-sm"
onClick={close}>{user.company || user.first_name}</Link>
          ) : (
            <button className="btn btn-green btn-sm" onClick={() =>
setModal('login')}>Кабинет покупателя</button>
          )}
          <button className="burger" aria-label="Меню" onClick={() =>
setOpen(!open)}><span /><span /><span /></button>
        </div>
      </div>
```

```
    </header>
  )
}
```

Текст файла src/components/Logo.jsx

```
export default function Logo({ light = false }) {
  const ink = light ? '#F4EFE3' : '#1F3A2B'
  return (
    <span className="logo">
      <svg viewBox="0 0 64 64" width="40" height="40" aria-hidden="true">
        <circle cx="32" cy="32" r="32" fill={light ? '#F4EFE3' : '#1F3A2B'} />
        <path d="M32 52V16" stroke="#E2B04A" strokeWidth="4" strokeLinecap="round" />
        <path d="M32 24c-6-1-10-5-10-10 6 1 10 5 10 10zM32 24c6-1 10-5 10-10-6 1-10 5-
10 10zM32 34c-6-1-10-5-10-10 6 1 10 5 10 10zM32 34c6-1 10-5 10-10-6 1-10 5-10 10zM32
44c-6-1-10-5-10-10 6 1 10 5 10 10zM32 44c6-1 10-5 10-10-6 1-10 5-10 10z" fill="#E2B04A"
/>
      </svg>
      <span className="logo-text" style={{ color: ink }}>
        Колос<span>Юг</span>
      </span>
    </span>
  )
}
```

Текст файла src/pages/About.jsx

```
import { img } from '../api.js'
import '../styles/about.css'


const history = [
  ['2004', 'Основано первое хозяйство в Тимашевском районе Краснодарского края, 3 200
га.'],
  ['2009', 'Построен Тимашевский элеватор на 60 тысяч тонн и собственная
лаборатория.'],
  ['2014', 'Вошли хозяйства в Ростовской области и Ставропольском крае.'],
  ['2019', 'Начато выращивание сои и сахарной свёклы в Черноземье.'],
  ['2026', '6 хозяйств, 87 700 га и продажи через информационную систему.'],
]
const values = [
  ['Земля', 'Севообороты, точное земледелие и сохранение плодородия чернозёмов.'],
  ['Люди', '1 400 сотрудников, обучение механизаторов и агрономов на базе холдинга.'],
  ['Качество', 'Контроль каждой партии от поля до вагона по ГОСТ.'],
]


export default function About() {
  return (
    <>
      <section className="about-hero">
        <div className="container about-hero-grid">
          <div>
            <p className="eyebrow">О холдинге</p>
            <h1>Работаем на земле больше 20 лет</h1>
            <p className="lead">«Колос-Юг» объединяет растениеводческие хозяйства,
элеваторы и логистику. Мы продаём зерно мукомольным, комбикормовым и
маслоэкстракционным заводам, а также экспортёрам.</p>
          </div>
          <img src={img('about.jpg')} alt="Элеватор холдинга" />
        </div>
      </section>
      <section className="section">
        <div className="container">
          <h2>История</h2>
          <ol className="timeline">
            {history.map(([y, t]) => <li key={y}><strong>{y}</strong><p>{t}</p></li>)}
          </ol>
```

```
        </div>
      </section>
      <section className="section section-cream">
        <div className="container">
          <h2>Ценности</h2>
          <div className="values">
            {values.map(([h, p]) => <div key={h}><h3>{h}</h3><p>{p}</p></div>)}
          </div>
        </div>
      </section>
      <section className="section">
        <div className="container tractor-band" style={{ backgroundImage:
`url(${img('tractor.jpg')})` }}>
          <div><h2>Собственный парк техники</h2><p>320 тракторов и комбайнов, система
мониторинга техники и спутниковые карты полей.</p></div>
        </div>
      </section>
    </>
  )
}
```

Текст файла src/pages/Account.jsx

```
import { useEffect, useState } from 'react'
import { Link, useNavigate } from 'react-router-dom'
import { get, post, img, money, num, date, STATUS } from '../api.js'
import { useAuth } from '../AuthContext.jsx'
import '../styles/account.css'


export default function Account() {
  const { user, logout } = useAuth()
  const navigate = useNavigate()
  const [orders, setOrders] = useState([])
  const [msg, setMsg] = useState({ type: '', text: '' })


  const load = () => get('myOrders.php').then((d) => setOrders(d.orders)).catch((e) =>
setMsg({ type: 'error', text: e.message }))
  useEffect(() => {
    load()
  }, [])


  const act = async (path, o, question) => {
    if (question && !window.confirm(question)) return
    try {
      const d = await post(path, { id: o.id_order })
      setMsg({ type: 'success', text: d.message })
      load()
    } catch (err) {
      setMsg({ type: 'error', text: err.message })
    }
  }


  const exit = async () => {
    await logout()
    navigate('/')
  }


  const active = orders.filter((o) => ['new', 'confirmed',
'shipped'].includes(o.status))
  const done = orders.filter((o) => o.status === 'completed')


  return (
    <div className="container account">
      <div className="account-head">
```

```
        <div>
          <p className="eyebrow">Кабинет покупателя</p>
          <h1>{user.company || user.first_name}</h1>
          <p className="muted">{user.last_name} {user.first_name} · ИНН {user.inn || '-
'} · {user.email} · {user.phone}</p>
        </div>
        <div className="account-actions">
          {['admin', 'manager'].includes(user.role) && <Link to="/admin" className="btn
btn-green btn-sm">Управление</Link>}
          <Link to="/products" className="btn btn-gold btn-sm">Новый заказ</Link>
          <button className="btn btn-line btn-sm" onClick={exit}>Выйти</button>
        </div>
      </div>
      <div className="account-stats">
        <div><strong>{orders.length}</strong><span>всего заказов</span></div>
        <div><strong>{active.length}</strong><span>в работе</span></div>
        <div><strong>{num(done.reduce((s, o) => s + Number(o.quantity_t), 0))}
т</strong><span>получено продукции</span></div>
        <div><strong>{money(orders.filter((o) => o.status !== 'cancelled').reduce((s,
o) => s + Number(o.total_price), 0))}</strong><span>сумма заказов</span></div>
      </div>
      {msg.text && <div className={'alert alert-' + msg.type}>{msg.text}</div>}
      <h2>Мои заказы</h2>
      {orders.length === 0 && <div className="empty">Заказов пока нет. <Link
to="/products">Перейти в каталог</Link></div>}
      <div className="orders">
        {orders.map((o) => (
          <article key={o.id_order} className="order">
            <img src={img(o.image)} alt="" />
            <div className="order-body">
              <div className="order-top">
                <span className="muted">Заказ № {o.id_order} от
{date(o.created_at)}</span>
                <span className={'status status-' + o.status}>{STATUS[o.status]}</span>
              </div>
              <h3><Link to={'/products/' + o.id_crop}>{o.crop}</Link></h3>
              <p className="muted">{num(o.quantity_t, 1)} т × {money(o.price_per_ton)}
· склад: {o.warehouse} · {o.delivery}{o.address ? ', ' + o.address : ''}</p>
              <div className="order-foot">
                <strong>{money(o.total_price)}</strong>
                <div className="order-btns">
                  {o.status === 'shipped' && <button className="btn btn-gold btn-sm"
onClick={() => act('confirmReceipt.php', o)}>Подтвердить получение</button>}
                  {['new', 'confirmed'].includes(o.status) && <button className="btn
btn-line btn-sm" onClick={() => act('cancelOrder.php', o, 'Отменить заказ № ' +
o.id_order + '?')}>Отменить</button>}
                </div>
              </div>
              {o.status === 'new' && <p className="hint">Заказ проверяет менеджер.
После подтверждения на почту придут договор и счёт.</p>}
            </div>
          </article>
        ))}
      </div>
    </div>
  )
}
```

Текст файла src/pages/Admin.jsx

```
import { useEffect, useState } from 'react'
import { get, post, money, num } from '../api.js'
import { useAuth } from '../AuthContext.jsx'
import '../styles/admin.css'

const LABELS = {
```

```
  id_user: 'ID клиента', id_farm: 'ID хозяйства', id_field: 'ID поля', id_crop: 'ID
культуры', id_sowing: 'ID посева', id_harvest: 'ID сбора',
  id_warehouse: 'ID склада', id_stock: 'ID остатка', id_order: 'ID заказа', id_request:
'ID заявки',
  last_name: 'Фамилия', first_name: 'Имя', company: 'Организация', inn: 'ИНН', email:
'E-mail', phone: 'Телефон', role: 'Роль',
  created_at: 'Создано', name: 'Название', region: 'Регион', district: 'Район',
area_ha: 'Площадь, га', specialization: 'Специализация',
  image: 'Изображение', cadastral_number: 'Кадастровый номер', soil_type: 'Почва',
category: 'Категория', grade: 'Класс',
  description: 'Описание', price_per_ton: 'Цена за т', is_active: 'Активна', season:
'Сезон', sown_date: 'Дата сева',
  planned_yield: 'План, ц/га', harvest_date: 'Дата уборки', amount_t: 'Собрано, т',
moisture: 'Влажность, %', quality_class: 'Качество',
  type: 'Тип', address: 'Адрес', capacity_t: 'Ёмкость, т', quantity_t: 'Количество, т',
updated_at: 'Обновлено', total_price: 'Сумма',
  delivery: 'Доставка', status: 'Статус', comment: 'Комментарий', topic: 'Тема',
message: 'Сообщение', password: 'Пароль',
  bought_t: 'Куплено, т', completion_percent: 'Выполнение, %', contact: 'Контакт',
crop: 'Культура', fact_t: 'Факт, т', farm: 'Хозяйство',
  fields_area_ha: 'Площадь полей, га', fields_count: 'Полей', load_percent: 'Загрузка,
%', month: 'Месяц', orders_count: 'Заказов',
  plan_t: 'План, т', revenue: 'Выручка', sold_t: 'Продано, т', stock_t: 'Остаток, т',
stored_t: 'Хранится, т', total_area_ha: 'Общая площадь, га',
  total_sum: 'Сумма заказов', warehouse: 'Склад', warehouses: 'Складов', yield_c_ha:
'Урожайность, ц/га',
}
const label = (k) => LABELS[k] || k
const short = (v) => {
  const s = v === null || v === undefined ? '-' : String(v)
  return s.length > 60 ? s.slice(0, 57) + '…' : s
}

function Field({ col, value, onChange }) {
  if (col.options) {
    return (
      <select value={value ?? ''} onChange={(e) => onChange(e.target.value)}>
        <option value="">-</option>
        {col.options.map((o) => <option key={o}>{o}</option>)}
      </select>
    )
  }
  const type = col.type === 'date' ? 'date' : ['int', 'tinyint', 'smallint',
'decimal'].includes(col.type) ? 'number' : col.type === 'datetime' ? 'text' : 'text'
  if (col.type === 'text') return <textarea rows="2" value={value ?? ''} onChange={(e)
=> onChange(e.target.value)} />
  return <input type={type} value={value ?? ''} onChange={(e) =>
onChange(e.target.value)} placeholder={col.type === 'datetime' ? 'ГГГГ-ММ-ДД ЧЧ:ММ:СС'
: ''} />
}

export default function Admin() {
  const { user } = useAuth()
  const [meta, setMeta] = useState({ tables: [], views: [] })
  const [stats, setStats] = useState(null)
  const [mode, setMode] = useState({ kind: 'table', name: 'orders' })
  const [data, setData] = useState(null)
  const [form, setForm] = useState({})
  const [editId, setEditId] = useState(null)
  const [msg, setMsg] = useState({ type: '', text: '' })

  useEffect(() => {
    get('admin/meta.php').then(setMeta).catch((e) => setMsg({ type: 'error', text:
e.message }))
    get('admin/stats.php').then(setStats).catch(() => {})
  }, [])
```

```
  const load = () => {
    setData(null)
    const url = mode.kind === 'table' ? 'admin/getTable.php?table=' + mode.name :
'admin/getView.php?view=' + mode.name
    get(url).then(setData).catch((e) => setMsg({ type: 'error', text: e.message }))
  }

  useEffect(() => {
    if (mode.name) load()
    setForm({})
    setEditId(null)
  }, [mode])

  const isTable = Boolean(data && data.columns)
  const editable = isTable ? data.columns.filter((c) => c.name !== data.pk &&
!c.extra.includes('auto_increment')) : []

  const startEdit = (row) => {
    setEditId(row[data.pk])
    setForm(row)
    setMsg({ type: '', text: '' })
  }

  const cancelEdit = () => {
    setEditId(null)
    setForm({})
  }

  const save = async () => {
    try {
      const d = editId
        ? await post('admin/updateRow.php', { table: mode.name, id: editId, row: form
})
        : await post('admin/insertRow.php', { table: mode.name, row: form })
      setMsg({ type: 'success', text: d.message })
      cancelEdit()
      load()
      get('admin/stats.php').then(setStats).catch(() => {})
    } catch (err) {
      setMsg({ type: 'error', text: err.message })
    }
  }

  const remove = async (row) => {
    if (!window.confirm('Удалить запись с ID ' + row[data.pk] + '?')) return
    try {
      const d = await post('admin/deleteRow.php', { table: mode.name, id: row[data.pk]
})
      setMsg({ type: 'success', text: d.message })
      load()
    } catch (err) {
      setMsg({ type: 'error', text: err.message })
    }
  }

  const columns = data ? (isTable ? data.columns.map((c) => c.name) :
Object.keys(data.rows[0] || {})) : []

  return (
    <div className="admin">
      <div className="container">
        <div className="admin-head">
```

```
          <div>
            <p className="eyebrow">{user.role === 'admin' ? 'Администратор' :
'Менеджер'}</p>
            <h1>Панель управления</h1>
          </div>
          {stats && (
            <div className="admin-stats">
              <div><strong>{stats.newOrders}</strong><span>новых заказов</span></div>
              <div><strong>{stats.newRequests}</strong><span>новых заявок</span></div>
              <div><strong>{num(stats.stock)} т</strong><span>на складах</span></div>
              <div><strong>{money(stats.revenue)}</strong><span>выручка</span></div>
            </div>
          )}
        </div>


        <div className="admin-pickers">
          <label>Таблица
            <select value={mode.kind === 'table' ? mode.name : ''} onChange={(e) =>
e.target.value && setMode({ kind: 'table', name: e.target.value })}>
              <option value="">- выберите -</option>
              {meta.tables.map((t) => <option key={t.name} value={t.name}>{t.title}
({t.name})</option>)}
            </select>
          </label>
          <label>Представление
            <select value={mode.kind === 'view' ? mode.name : ''} onChange={(e) =>
e.target.value && setMode({ kind: 'view', name: e.target.value })}>
              <option value="">- выберите -</option>
              {meta.views.map((v, i) => <option key={v.name} value={v.name}>№{i + 1}.
{v.title}</option>)}
            </select>
          </label>
        </div>


        {msg.text && <div className={'alert alert-' + msg.type}>{msg.text}</div>}


        {data && (
          <>
            <h2>{data.title} <small>{data.table || data.view} · записей:
{data.rows.length}</small></h2>
            <div className="table-wrap">
              <table className="table admin-table">
                <thead>
                  <tr>
                    {columns.map((c) => <th key={c}>{label(c)}</th>)}
                    {isTable && <th>Действия</th>}
                  </tr>
                </thead>
                <tbody>
                  {data.rows.map((r, i) => (
                    <tr key={isTable ? r[data.pk] : i} className={isTable && editId ===
r[data.pk] ? 'editing' : ''}>
                      {columns.map((c) => <td key={c} title={r[c] ??
''}>{short(r[c])}</td>)}
                      {isTable && (
                        <td className="actions">
                          <button className="btn-mini btn-edit" onClick={() =>
startEdit(r)}>Редактировать</button>
                          <button className="btn-mini btn-del" onClick={() =>
remove(r)}>Удалить</button>
                        </td>
                      )}
                    </tr>
                  ))}
                </tbody>
              </table>
```

```
            </div>

            {isTable && (
              <div className="admin-form">
                <h3>{editId ? 'Редактирование записи ID ' + editId : 'Добавление
записи'}</h3>
                <div className="admin-form-grid">
                  {editable.map((c) => (
                    <label key={c.name}>{label(c.name)}{c.nullable === 'NO' && c.def
=== null ? '*' : ''}
                      <Field col={c} value={form[c.name]} onChange={(v) => setForm({
...form, [c.name]: v })} />
                    </label>
                  ))}
                  {data.needsPassword && (
                    <label>{editId ? 'Новый пароль (не обязательно)' : 'Пароль*'}
                      <input type="password" value={form.password ?? ''} onChange={(e)
=> setForm({ ...form, password: e.target.value })} />
                    </label>
                  )}
                </div>
                <div className="admin-form-btns">
                  <button className={'btn btn-sm ' + (editId ? 'btn-gold' : 'btn-
green')} onClick={save}>{editId ? 'Сохранить' : 'Добавить'}</button>
                  {editId && <button className="btn btn-sm btn-line"
onClick={cancelEdit}>Отмена</button>}
                </div>
              </div>
            )}
          </>
        )}
      </div>
    </div>
  )
}
```

Текст файла src/pages/Contacts.jsx

```
import CallForm from '../components/CallForm.jsx'
import '../styles/about.css'

const offices = [
  ['Головной офис', 'Краснодар, ул. Красная, 120', '+7 (861) 200-10-00'],
  ['Отдел продаж зерна', 'sales@kolos-yug.ru', '+7 (861) 200-10-05'],
  ['Логистика и отгрузки', 'logistic@kolos-yug.ru', '+7 (861) 200-10-07'],
]

export default function Contacts() {
  return (
    <>
      <div className="container contacts">
        <div className="page-title"><p className="eyebrow">Связаться с
нами</p><h1>Контакты</h1></div>
        <div className="contacts-grid">
          <div className="contacts-info">
            {offices.map(([h, a, p]) => (
              <div key={h}><span>{h}</span><strong>{a}</strong><a href={'tel:' +
p.replace(/[^\d+]/g, '')}>{p}</a></div>
            ))}
            <div><span>Режим работы</span><strong>Пн-Пт 8:00-17:00</strong></div>
          </div>
          <iframe className="map" title="Офис на карте" src="https://yandex.ru/map-
widget/v1/?ll=38.975000%2C45.039000&z=16&pt=38.975000,45.039000,pm2grm" loading="lazy"
/>
        </div>
      </div>
```

```
      <CallForm />
    </>
  )
}
```

Текст файла src/pages/Farms.jsx

```
import { useEffect, useState } from 'react'
import { get, img, num } from '../api.js'
import '../styles/about.css'


export default function Farms() {
  const [farms, setFarms] = useState([])
  useEffect(() => {
    get('farms.php').then((d) => setFarms(d.farms)).catch(() => {})
  }, [])
  const total = farms.reduce((s, f) => s + Number(f.area_ha), 0)


  return (
    <div className="container farms-page">
      <div className="page-title">
        <p className="eyebrow">Производство</p>
        <h1>Хозяйства холдинга</h1>
        <p className="muted">{farms.length} хозяйств общей площадью {num(total)} га в
Краснодарском и Ставропольском краях, Ростовской, Воронежской, Белгородской и Курской
областях.</p>
      </div>
      <div className="farms-list">
        {farms.map((f) => (
          <article key={f.id_farm} className="farm-card">
            <div className="farm-card-img" style={{ backgroundImage:
`url(${img(f.image)})` }} />
            <div className="farm-card-body">
              <span className="muted">{f.region}, {f.district}</span>
              <h2>{f.name}</h2>
              <p>{f.specialization}</p>
              <dl>
                <div><dt>Площадь</dt><dd>{num(f.area_ha)} га</dd></div>
                <div><dt>Полей в учёте</dt><dd>{f.fields_count}</dd></div>
                <div><dt>Урожай 2026</dt><dd>{num(f.harvest_t)} т</dd></div>
                <div><dt>Хранение</dt><dd>{num(f.capacity_t)} т</dd></div>
              </dl>
            </div>
          </article>
        ))}
      </div>
    </div>
  )
}
```

Текст файла src/pages/Home.jsx

```
import { useEffect, useState } from 'react'
import { Link } from 'react-router-dom'
import { get, img, num } from '../api.js'
import CropCard from '../components/CropCard.jsx'
import CallForm from '../components/CallForm.jsx'
import '../styles/home.css'


const steps = [
  ['Выберите продукцию', 'В каталоге указаны цена за тонну, класс качества и остатки на
каждом элеваторе.'],
  ['Оформите заказ', 'Укажите объём, склад отгрузки и способ доставки. Продукция
резервируется сразу.'],
  ['Получите счёт', 'Менеджер подтвердит заказ, пришлёт договор, счёт и протокол
испытаний партии.'],
```

```
  ['Заберите партию', 'Самовывоз с элеватора, доставка автотранспортом или отгрузка по
железной дороге.'],
]

const features = [
  ['Полный цикл', 'От обработки почвы до хранения и отгрузки: всё под контролем одной
компании.'],
  ['Своё хранение', '7 элеваторов и складов общей ёмкостью 288 тысяч тонн.'],
  ['Контроль качества', 'Лаборатории на каждом элеваторе, протоколы по ГОСТ на каждую
партию.'],
  ['Прозрачные цены', 'Цена за тонну и наличие на складах обновляются в каталоге
ежедневно.'],
]

export default function Home() {
  const [stats, setStats] = useState(null)
  const [crops, setCrops] = useState([])
  const [farms, setFarms] = useState([])

  useEffect(() => {
    get('stats.php').then(setStats).catch(() => {})
    get('crops.php?inStock=1&sort=stock').then((d) => setCrops(d.crops.slice(0,
6))).catch(() => {})
    get('farms.php').then((d) => setFarms(d.farms)).catch(() => {})
  }, [])

  return (
    <>
      <section className="hero">
        <div className="container hero-grid">
          <div className="hero-text">
            <p className="eyebrow">Агрохолдинг «Колос-Юг»</p>
            <h1>Зерно и масличные напрямую от производителя</h1>
            <p className="lead">Выращиваем, храним и отгружаем пшеницу, ячмень,
кукурузу, подсолнечник, сою и рапс с собственных элеваторов на юге России.</p>
            <div className="hero-btns">
              <Link to="/products" className="btn btn-gold">Каталог продукции</Link>
              <a href="#callform" className="btn btn-line">Оставить заявку</a>
            </div>
          </div>
          <div className="hero-img" style={{ backgroundImage: `url(${img('hero.jpg')})`
}} />
        </div>
      </section>

      <section className="stats">
        <div className="container stats-grid">
          <div><strong>{stats ? stats.farms : '-'}</strong><span>хозяйств в 6
регионах</span></div>
          <div><strong>{stats ? num(stats.area) : '-'}</strong><span>гектаров
земли</span></div>
          <div><strong>{stats ? num(stats.harvest) : '-'}</strong><span>тонн собрано в
2026 году</span></div>
          <div><strong>{stats ? num(stats.capacity) : '-'}</strong><span>тонн ёмкость
хранения</span></div>
        </div>
      </section>

      <section className="section">
        <div className="container">
          <div className="section-head">
            <h2>Продукция в наличии</h2>
            <Link to="/products" className="link-arrow">Весь каталог →</Link>
          </div>
```

```
          <div className="crops-grid">
            {crops.map((c) => <CropCard key={c.id_crop} crop={c} />)}
          </div>
        </div>
      </section>

      <section className="section section-cream">
        <div className="container">
          <div className="section-head">
            <h2>Наши хозяйства</h2>
            <Link to="/farms" className="link-arrow">Подробнее →</Link>
          </div>
          <div className="farms-strip">
            {farms.map((f) => (
              <div key={f.id_farm} className="farm-mini" style={{ backgroundImage:
`url(${img(f.image)})` }}>
                <div>
                  <span>{f.region}</span>
                  <h3>{f.name}</h3>
                  <small>{num(f.area_ha)} га</small>
                </div>
              </div>
            ))}
          </div>
        </div>
      </section>

      <section className="section">
        <div className="container two-col">
          <div>
            <h2>Почему покупают у нас</h2>
            <div className="features">
              {features.map(([h, p]) => <div key={h}
className="feature"><h3>{h}</h3><p>{p}</p></div>)}
            </div>
          </div>
          <div>
            <h2>Как купить</h2>
            <ol className="steps">
              {steps.map(([h, p], i) => <li key={h}><span>{i +
1}</span><div><h3>{h}</h3><p>{p}</p></div></li>)}
            </ol>
          </div>
        </div>
      </section>

      <CallForm />
    </>
  )
}
```

Текст файла src/pages/NotFound.jsx

```
import { Link } from 'react-router-dom'

export default function NotFound() {
  return (
    <div className="container page-msg">
      <h1>Страница не найдена</h1>
      <p>Возможно, ссылка устарела или страница была перемещена.</p>
      <Link to="/" className="btn btn-green">На главную</Link>
    </div>
  )
}
```

Текст файла src/pages/Product.jsx

```
import { useEffect, useState } from 'react'
import { Link, useNavigate, useParams } from 'react-router-dom'
import { get, post, img, money, num } from '../api.js'
import { useAuth } from '../AuthContext.jsx'
import '../styles/product.css'


export default function Product() {
  const { id } = useParams()
  const navigate = useNavigate()
  const { user, setModal } = useAuth()
  const [crop, setCrop] = useState(null)
  const [error, setError] = useState('')
  const [form, setForm] = useState({ warehouseId: '', quantity: 50, delivery:
'Самовывоз', address: '', comment: '' })
  const [msg, setMsg] = useState({ type: '', text: '' })
  const [busy, setBusy] = useState(false)


  const load = () =>
    get('crop.php?id=' + id)
      .then((d) => {
        setCrop(d.crop)
        const first = d.crop.stock.find((s) => Number(s.quantity_t) > 0)
        setForm((f) => ({ ...f, warehouseId: first ? String(first.id_warehouse) : ''
}))
      })
      .catch((e) => setError(e.message))


  useEffect(() => {
    load()
  }, [id])


  if (error) return <div className="container page-msg"><h1>{error}</h1><Link
to="/products" className="btn btn-green">В каталог</Link></div>
  if (!crop) return <div className="page-loader">Загрузка...</div>


  const set = (k) => (e) => setForm({ ...form, [k]: e.target.value })
  const wh = crop.stock.find((s) => String(s.id_warehouse) === form.warehouseId)
  const qty = Number(form.quantity) || 0
  const total = qty * Number(crop.price_per_ton)
  const totalStock = crop.stock.reduce((s, x) => s + Number(x.quantity_t), 0)


  const submit = async (e) => {
    e.preventDefault()
    if (!user) return setModal('login')
    if (user.role !== 'client') return setMsg({ type: 'error', text: 'Заказ может
оформить только покупатель' })
    if (!wh) return setMsg({ type: 'error', text: 'Выберите склад отгрузки' })
    if (qty < 5 || qty > 5000) return setMsg({ type: 'error', text: 'Объём заказа
должен быть от 5 до 5 000 тонн' })
    if (qty > Number(wh.quantity_t)) return setMsg({ type: 'error', text: 'Недостаточно
продукции на складе. Доступно: ' + num(wh.quantity_t, 1) + ' т' })
    if (form.delivery !== 'Самовывоз' && form.address.trim().length < 10) return
setMsg({ type: 'error', text: 'Укажите адрес доставки' })
    setBusy(true)
    try {
      const d = await post('order.php', { cropId: crop.id_crop, ...form, quantity: qty
})
      setMsg({ type: 'success', text: d.message })
      await load()
      setTimeout(() => navigate('/account'), 1500)
    } catch (err) {
      setMsg({ type: 'error', text: err.message })
```

```
    } finally {
      setBusy(false)
    }
  }

  return (
    <div className="container product">
      <Link to="/products" className="back">← Вся продукция</Link>
      <div className="product-top">
        <div className="product-img"><img src={img(crop.image)} alt={crop.name}
/></div>
        <div className="product-info">
          <span className="chip">{crop.category}</span>
          <h1>{crop.name}</h1>
          <p className="grade">{crop.grade}</p>
          <p>{crop.description}</p>
          <div className="price-row">
            <div><small>Цена за
тонну</small><strong>{money(crop.price_per_ton)}</strong></div>
            <div><small>Всего на складах</small><strong>{num(totalStock, 1)}
т</strong></div>
          </div>
        </div>
      </div>

      <div className="product-grid">
        <div>
          <h2>Наличие на складах</h2>
          {crop.stock.length === 0 && <p className="muted">Продукция временно
отсутствует на складах.</p>}
          {crop.stock.length > 0 && (
            <table className="table">
              <thead><tr><th>Склад</th><th>Регион</th><th>Адрес</th><th>Остаток,
т</th></tr></thead>
              <tbody>
                {crop.stock.map((s) => (
                  <tr key={s.id_warehouse} className={String(s.id_warehouse) ===
form.warehouseId ? 'active' : ''} onClick={() => Number(s.quantity_t) > 0 && setForm({
...form, warehouseId: String(s.id_warehouse) })}>
                    <td><b>{s.name}</b><br /><small
className="muted">{s.type}</small></td>
                    <td>{s.region}</td>
                    <td>{s.address}</td>
                    <td>{num(s.quantity_t, 1)}</td>
                  </tr>
                ))}
              </tbody>
            </table>
          )}
          <h2>Производство по сезонам</h2>
          <table className="table">
            <thead><tr><th>Сезон</th><th>Посевная площадь, га</th><th>Собрано,
т</th><th>Урожайность, ц/га</th></tr></thead>
            <tbody>
              {crop.seasons.map((s) => (
                <tr key={s.season}>
                  <td>{s.season}</td>
                  <td>{num(s.area_ha, 1)}</td>
                  <td>{Number(s.amount_t) > 0 ? num(s.amount_t, 1) : 'уборка
идёт'}</td>
                  <td>{Number(s.amount_t) > 0 ? num((s.amount_t * 10) / s.area_ha, 1) :
'-'}</td>
                </tr>
              ))}
            </tbody>
          </table>
```

```
        </div>

        <aside className="order-box">
          <h3>Оформление заказа</h3>
          <form onSubmit={submit} noValidate>
            <label>Склад отгрузки
              <select value={form.warehouseId} onChange={set('warehouseId')}>
                {crop.stock.filter((s) => Number(s.quantity_t) > 0).map((s) => (
                  <option key={s.id_warehouse} value={s.id_warehouse}>{s.name}
({num(s.quantity_t)} т)</option>
                ))}
              </select>
            </label>
            <label>Объём, тонн
              <input type="number" min="5" max="5000" step="1" value={form.quantity}
onChange={set('quantity')} />
            </label>
            <label>Доставка
              <select value={form.delivery} onChange={set('delivery')}>
                <option>Самовывоз</option>
                <option>Автодоставка</option>
                <option>Железная дорога</option>
              </select>
            </label>
            {form.delivery !== 'Самовывоз' && (
              <label>Адрес доставки<input value={form.address}
onChange={set('address')} placeholder="Город, улица, дом или станция" /></label>
            )}
            <label>Комментарий<textarea rows="2" value={form.comment}
onChange={set('comment')} placeholder="Сроки, требования к качеству" /></label>
            <div className="order-
total"><span>Сумма</span><strong>{money(total)}</strong></div>
            {msg.text && <div className={'alert alert-' + msg.type}>{msg.text}</div>}
            <button className="btn btn-gold btn-block" disabled={busy || !wh}>{user ?
'Оформить заказ' : 'Войти и оформить заказ'}</button>
          </form>
        </aside>
      </div>
    </div>
  )
}
```

Текст файла src/pages/Products.jsx

```
import { useEffect, useState } from 'react'
import { useSearchParams } from 'react-router-dom'
import { get, CATEGORIES } from '../api.js'
import CropCard from '../components/CropCard.jsx'
import '../styles/catalog.css'

const FIELDS = ['q', 'category', 'priceMax', 'inStock', 'sort']

export default function Products() {
  const [params, setParams] = useSearchParams()
  const [crops, setCrops] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState('')
  const f = Object.fromEntries(FIELDS.map((k) => [k, params.get(k) || '']))

  useEffect(() => {
    setLoading(true)
    setError('')
    get('crops.php?' + params.toString())
      .then((d) => setCrops(d.crops))
      .catch((e) => setError(e.message))
```

```
      .finally(() => setLoading(false))
  }, [params])

  const update = (key, value) => {
    const next = new URLSearchParams(params)
    if (value) next.set(key, value)
    else next.delete(key)
    setParams(next)
  }

  return (
    <div className="container catalog">
      <div className="page-title">
        <p className="eyebrow">Оптовые продажи</p>
        <h1>Продукция</h1>
        <p className="muted">Цены указаны за тонну с НДС на условиях самовывоза с
элеватора. Минимальная партия 5 тонн.</p>
      </div>
      <div className="cat-tabs">
        <button className={!f.category ? 'on' : ''} onClick={() => update('category',
'')}>Все</button>
        {CATEGORIES.map((c) => <button key={c} className={f.category === c ? 'on' : ''}
onClick={() => update('category', c)}>{c}</button>)}
      </div>
      <div className="catalog-layout">
        <aside className="filters">
          <label>Поиск
            <input key={"q" + f.q} placeholder="Культура или класс" defaultValue={f.q}
onKeyDown={(e) => e.key === 'Enter' && update('q', e.target.value.trim())} onBlur={(e)
=> update('q', e.target.value.trim())} />
          </label>
          <label>Цена до,  за тонну
            <input key={"p" + f.priceMax} type="number" min="0" step="1000"
defaultValue={f.priceMax} onBlur={(e) => update('priceMax', e.target.value)}
onKeyDown={(e) => e.key === 'Enter' && update('priceMax', e.target.value)} />
          </label>
          <label className="check"><input type="checkbox" checked={f.inStock === '1'}
onChange={(e) => update('inStock', e.target.checked ? '1' : '')} /> Только в
наличии</label>
          <label>Сортировка
            <select value={f.sort} onChange={(e) => update('sort', e.target.value)}>
              <option value="">Сначала дешёвые</option>
              <option value="price_desc">Сначала дорогие</option>
              <option value="stock">По остаткам</option>
              <option value="name">По названию</option>
            </select>
          </label>
          <button className="btn btn-line btn-block btn-sm" onClick={() =>
setParams({})}>Сбросить</button>
        </aside>
        <section>
          <p className="found">{loading ? 'Загрузка...' : 'Найдено позиций: ' +
crops.length}</p>
          {error && <div className="alert alert-error">{error}</div>}
          {!loading && !error && crops.length === 0 && <div className="empty">По
выбранным условиям продукции нет. Измените фильтры или оставьте заявку.</div>}
          <div className="crops-grid crops-grid-3">
            {crops.map((c) => <CropCard key={c.id_crop} crop={c} />)}
          </div>
        </section>
      </div>
    </div>
  )
}
```

Текст файла src/styles/about.css

```
.farms-page { padding-bottom: 72px; }
.farms-list { display: flex; flex-direction: column; gap: 28px; }
.farm-card { display: grid; grid-template-columns: 420px 1fr; background: #fff; border:
1px solid var(--line); border-radius: var(--radius); overflow: hidden; }
.farm-card:nth-child(even) { grid-template-columns: 1fr 420px; }
.farm-card:nth-child(even) .farm-card-img { order: 2; }
.farm-card-img { min-height: 280px; background: var(--green-3) center/cover; }
.farm-card-body { padding: 30px 34px; }
.farm-card-body h2 { font-size: 28px; margin: 6px 0 10px; }
.farm-card-body dl { display: grid; grid-template-columns: repeat(4, 1fr); margin: 20px
0 0; border-top: 1px solid var(--line); }
.farm-card-body dl div { padding-top: 14px; }
.farm-card-body dt { font-size: 13px; color: var(--muted); }
.farm-card-body dd { margin: 0; font: 700 20px var(--serif); color: var(--green); }
.about-hero { background: var(--cream); padding: 64px 0; }
.about-hero-grid { display: grid; grid-template-columns: 1.1fr 1fr; gap: 56px; align-
items: center; }
.about-hero img { border-radius: var(--radius); aspect-ratio: 4 / 3; object-fit: cover;
width: 100%; box-shadow: -18px 18px 0 var(--green); }
.timeline { list-style: none; padding: 0; margin: 0; display: grid; grid-template-
columns: repeat(5, 1fr); gap: 0; border-top: 3px solid var(--gold); }
.timeline li { padding: 20px 20px 0 0; position: relative; }
.timeline li::before { content: ''; position: absolute; top: -10px; left: 0; width:
16px; height: 16px; border-radius: 50%; background: var(--green); border: 3px solid
var(--gold); }
.timeline strong { display: block; font: 700 28px var(--serif); color: var(--green);
margin: 8px 0 6px; }
.timeline p { font-size: 15px; color: #4A544D; }
.values { display: grid; grid-template-columns: repeat(3, 1fr); gap: 28px; }
.values div { background: #fff; padding: 28px; border-radius: var(--radius); border-
left: 5px solid var(--gold); }
.values p { margin: 0; color: #4A544D; }
.tractor-band { min-height: 320px; background: var(--green-3) center/cover; border-
radius: var(--radius); display: flex; align-items: flex-end; padding: 0; overflow:
hidden; }
.tractor-band > div { background: rgba(31, 58, 43, .9); color: var(--cream); padding:
28px 32px; max-width: 520px; }
.tractor-band h2 { color: var(--gold); font-size: 26px; }
.tractor-band p { margin: 0; }
.contacts { padding-bottom: 56px; }
.contacts-grid { display: grid; grid-template-columns: 380px 1fr; gap: 32px; }
.contacts-info { display: flex; flex-direction: column; gap: 0; background: var(--
cream); border-radius: var(--radius); padding: 8px 26px; }
.contacts-info div { display: flex; flex-direction: column; padding: 16px 0; border-
bottom: 1px solid var(--line); }
.contacts-info div:last-child { border: 0; }
.contacts-info span { font-size: 12px; text-transform: uppercase; letter-spacing: .1em;
color: var(--gold-2); font-weight: 600; }
.contacts-info a { font-weight: 600; }
.map { width: 100%; height: 440px; border: 0; border-radius: var(--radius); background:
var(--cream); }
@media (max-width: 900px) {
  .farm-card, .farm-card:nth-child(even), .about-hero-grid, .contacts-grid { grid-
template-columns: 1fr; }
  .farm-card:nth-child(even) .farm-card-img { order: 0; }
  .farm-card-body dl { grid-template-columns: 1fr 1fr; }
  .timeline, .values { grid-template-columns: 1fr; }
}
```

Текст файла src/styles/account.css

```
.account { padding: 40px 24px 72px; }
.account-head { display: flex; justify-content: space-between; align-items: flex-end;
gap: 24px; flex-wrap: wrap; padding-bottom: 24px; border-bottom: 1px solid var(--line);
}
.account-head h1 { margin-bottom: 6px; font-size: 36px; }
.account-head p { margin: 0; }
```

```
.account-actions { display: flex; gap: 10px; flex-wrap: wrap; }
.account-stats { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px;
margin: 28px 0; }
.account-stats div { background: var(--cream); border-radius: var(--radius); padding:
18px 20px; }
.account-stats strong { display: block; font: 700 26px var(--serif); color: var(--
green); }
.account-stats span { font-size: 13px; color: var(--muted); }
.account h2 { font-size: 26px; margin-top: 12px; }
.orders { display: flex; flex-direction: column; gap: 16px; }
.order { display: grid; grid-template-columns: 150px 1fr; background: #fff; border: 1px
solid var(--line); border-radius: var(--radius); overflow: hidden; }
.order > img { width: 100%; height: 100%; object-fit: cover; min-height: 140px; }
.order-body { padding: 18px 22px; }
.order-top { display: flex; justify-content: space-between; align-items: center; font-
size: 14px; }
.order-body h3 { margin: 6px 0 4px; }
.order-body h3 a { color: var(--green); }
.order-body > p { font-size: 14px; margin-bottom: 10px; }
.order-foot { display: flex; justify-content: space-between; align-items: center; gap:
12px; flex-wrap: wrap; }
.order-foot strong { font: 700 22px var(--serif); color: var(--green); }
.order-btns { display: flex; gap: 8px; }
.hint { font-size: 13px; color: var(--muted); margin: 10px 0 0; padding-top: 10px;
border-top: 1px dashed var(--line); }
@media (max-width: 900px) { .account-stats { grid-template-columns: 1fr 1fr; } .order {
grid-template-columns: 1fr; } .order > img { height: 160px; } }
```

Текст файла src/styles/admin.css

```
.admin { padding: 36px 0 72px; background: #F2EFE7; }
.admin-head { display: flex; justify-content: space-between; align-items: flex-end;
gap: 24px; flex-wrap: wrap; margin-bottom: 24px; }
.admin-head h1 { margin: 0; font-size: 34px; }
.admin-stats { display: flex; gap: 12px; flex-wrap: wrap; }
.admin-stats div { background: var(--green); color: var(--cream); padding: 12px 18px;
border-radius: var(--radius); min-width: 130px; }
.admin-stats strong { display: block; font: 700 22px var(--serif); color: var(--gold);
}
.admin-stats span { font-size: 12px; }
.admin-pickers { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; background:
#fff; padding: 18px 20px; border-radius: var(--radius); border: 1px solid var(--line);
margin-bottom: 20px; }
.table-wrap { overflow-x: auto; background: #fff; border: 1px solid var(--line);
border-radius: var(--radius); margin-bottom: 24px; }
.admin-table { margin: 0; font-size: 13px; }
.admin-table th { white-space: nowrap; padding: 9px 12px; }
.admin-table td { padding: 8px 12px; }
.admin-table tr.editing td { background: #FFF3D6 !important; }
.actions { white-space: nowrap; }
.btn-mini { border: 0; border-radius: 4px; padding: 5px 10px; font: 600 12px var(--
sans); cursor: pointer; margin-right: 6px; }
.btn-edit { background: var(--gold); color: var(--green); }
.btn-del { background: #F1E1DD; color: var(--red); }
.admin-form { background: #fff; border: 1px solid var(--line); border-top: 5px solid
var(--gold); border-radius: var(--radius); padding: 22px 24px; }
.admin-form h3 { margin-bottom: 16px; }
.admin-form-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 14px; }
.admin-form-btns { display: flex; gap: 10px; margin-top: 18px; }
@media (max-width: 900px) { .admin-pickers, .admin-form-grid { grid-template-columns:
1fr; } }
```

Текст файла src/styles/auth.css

```
.modal-backdrop { position: fixed; inset: 0; background: rgba(20, 32, 24, .6); display:
flex; align-items: center; justify-content: center; z-index: 100; padding: 20px; }
```

```
.modal { position: relative; background: var(--paper); width: 100%; max-width: 480px;
max-height: 92vh; overflow: auto; border-radius: var(--radius); padding: 36px 36px
28px; border-top: 5px solid var(--gold); }
.modal h2 { font-size: 28px; }
.modal form { display: flex; flex-direction: column; gap: 14px; }
.modal-close { position: absolute; top: 12px; right: 16px; background: none; border: 0;
font-size: 28px; color: var(--muted); cursor: pointer; line-height: 1; }
.modal-switch { margin: 18px 0 0; text-align: center; font-size: 14px; color: var(--
muted); }
.modal-switch button { background: none; border: 0; color: var(--green-2); font-weight:
600; cursor: pointer; font-size: 14px; text-decoration: underline; }
```

Текст файла src/styles/base.css

```
:root {
  --green: #1F3A2B;
  --green-2: #2E5440;
  --green-3: #4F7A5E;
  --gold: #E2B04A;
  --gold-2: #C9962E;
  --cream: #F4EFE3;
  --paper: #FBF9F4;
  --ink: #1D241F;
  --muted: #6B736C;
  --line: #E1DACB;
  --red: #B3412E;
  --ok: #2F7D4A;
  --radius: 6px;
  --serif: 'Lora', Georgia, serif;
  --sans: 'Inter', Arial, sans-serif;
}
* { box-sizing: border-box; }
html { scroll-behavior: smooth; }
body { margin: 0; font-family: var(--sans); font-size: 16px; line-height: 1.6; color:
var(--ink); background: var(--paper); }
#root { min-height: 100vh; display: flex; flex-direction: column; }
main { flex: 1; }
h1, h2, h3, h4 { font-family: var(--serif); color: var(--green); line-height: 1.2;
margin: 0 0 .6em; font-weight: 700; }
h1 { font-size: 44px; }
h2 { font-size: 32px; }
h3 { font-size: 20px; font-weight: 600; }
p { margin: 0 0 1em; }
a { color: var(--green-2); text-decoration: none; }
a:hover { color: var(--gold-2); }
img { max-width: 100%; display: block; }
.container { width: 100%; max-width: 1200px; margin: 0 auto; padding: 0 24px; }
.section { padding: 72px 0; }
.section-cream { background: var(--cream); }
.section-head { display: flex; justify-content: space-between; align-items: baseline;
gap: 16px; margin-bottom: 28px; }
.section-head h2 { margin: 0; }
.link-arrow { font-weight: 600; }
.eyebrow { text-transform: uppercase; letter-spacing: .14em; font-size: 12px; font-
weight: 600; color: var(--gold-2); margin-bottom: 10px; }
.lead { font-size: 19px; color: #3C463F; }
.muted { color: var(--muted); }
.page-title { padding: 48px 0 24px; max-width: 760px; }
.page-title h1 { margin-bottom: 10px; }
.page-loader, .page-msg { padding: 120px 24px; text-align: center; color: var(--muted);
}
.page-msg h1 { font-size: 34px; }
.btn { display: inline-flex; align-items: center; justify-content: center; gap: 8px;
padding: 13px 26px; border-radius: var(--radius); border: 2px solid transparent; font:
600 15px/1.2 var(--sans); cursor: pointer; transition: .2s; text-align: center; }
.btn:disabled { opacity: .55; cursor: default; }
.btn-sm { padding: 9px 18px; font-size: 14px; }
```

```
.btn-block { width: 100%; }
.btn-green { background: var(--green); color: var(--cream); }
.btn-green:hover { background: var(--green-2); color: #fff; }
.btn-gold { background: var(--gold); color: var(--green); }
.btn-gold:hover { background: var(--gold-2); color: var(--green); }
.btn-line { background: transparent; border-color: var(--green); color: var(--green); }
.btn-line:hover { background: var(--green); color: var(--cream); }
.chip { display: inline-block; padding: 3px 10px; border-radius: 3px; background: var(-
-cream); color: var(--green); font-size: 12px; font-weight: 600; letter-spacing: .03em;
}
label { display: flex; flex-direction: column; gap: 6px; font-size: 14px; font-weight:
500; color: #3C463F; }
input, select, textarea { font: 15px var(--sans); padding: 11px 13px; border: 1px solid
var(--line); border-radius: var(--radius); background: #fff; color: var(--ink); width:
100%; }
input:focus, select:focus, textarea:focus { outline: 2px solid var(--gold); outline-
offset: -1px; border-color: var(--gold); }
textarea { resize: vertical; }
label.check { flex-direction: row; align-items: center; gap: 10px; font-weight: 400; }
label.check input { width: 18px; height: 18px; accent-color: var(--green); }
.row2 { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
.alert { padding: 12px 16px; border-radius: var(--radius); font-size: 14px; margin: 4px
0 12px; border-left: 4px solid; }
.alert-error { background: #FBEDEA; color: var(--red); border-color: var(--red); }
.alert-success { background: #E9F4EC; color: var(--ok); border-color: var(--ok); }
.empty { padding: 32px; text-align: center; background: var(--cream); border-radius:
var(--radius); color: var(--muted); }
.table { width: 100%; border-collapse: collapse; background: #fff; font-size: 14px;
margin-bottom: 32px; }
.table th { text-align: left; background: var(--green); color: var(--cream); font-
weight: 600; padding: 11px 14px; }
.table td { padding: 11px 14px; border-bottom: 1px solid var(--line); vertical-align:
top; }
.table tbody tr:nth-child(even) td { background: #FAF7EF; }
.status { display: inline-block; padding: 3px 10px; border-radius: 3px; font-size:
12px; font-weight: 600; }
.status-new { background: #FFF3D6; color: #8A6514; }
.status-confirmed { background: #E3EEF7; color: #2C5878; }
.status-shipped { background: #E7E3F5; color: #514088; }
.status-completed { background: #E2F1E6; color: var(--ok); }
.status-cancelled { background: #F1E6E3; color: var(--red); }
@media (max-width: 900px) {
  h1 { font-size: 34px; }
  h2 { font-size: 26px; }
  .section { padding: 48px 0; }
  .row2 { grid-template-columns: 1fr; }
}
```

Текст файла src/styles/callform.css

```
.callform { background: var(--green); padding: 72px 0; color: var(--cream); }
.callform-inner { display: grid; grid-template-columns: 1fr 1.3fr; gap: 56px; align-
items: start; }
.callform-text h2 { color: var(--cream); }
.callform-text p { color: #C9D2CB; }
.callform-text a { color: var(--gold); }
.callform-form { display: flex; flex-direction: column; gap: 14px; background: var(--
paper); padding: 32px; border-radius: var(--radius); border-top: 5px solid var(--gold);
}
.callform-form .btn { align-self: flex-start; }
@media (max-width: 900px) { .callform-inner { grid-template-columns: 1fr; gap: 28px; }
}
.callform-text ul { padding-left: 20px; margin: 0; } .callform-text li { margin-bottom:
6px; } .callform-text li::marker { color: var(--gold); }
```

Текст файла src/styles/catalog.css

```
.catalog { padding-bottom: 72px; }
.cat-tabs { display: flex; gap: 6px; flex-wrap: wrap; border-bottom: 2px solid var(--
line); margin-bottom: 28px; }
.cat-tabs button { background: none; border: 0; padding: 12px 18px; font: 500 15px
var(--sans); color: var(--muted); cursor: pointer; border-bottom: 3px solid
transparent; margin-bottom: -2px; }
.cat-tabs button:hover { color: var(--green); }
.cat-tabs button.on { color: var(--green); border-bottom-color: var(--gold); font-
weight: 600; }
.catalog-layout { display: grid; grid-template-columns: 260px 1fr; gap: 32px; align-
items: start; }
.filters { display: flex; flex-direction: column; gap: 16px; background: var(--cream);
padding: 22px; border-radius: var(--radius); position: sticky; top: 130px; }
.found { color: var(--muted); font-size: 14px; margin-bottom: 16px; }
.crops-grid-3 { grid-template-columns: repeat(3, 1fr); gap: 22px; }
@media (max-width: 1100px) { .crops-grid-3 { grid-template-columns: 1fr 1fr; } }
@media (max-width: 900px) { .catalog-layout { grid-template-columns: 1fr; } .filters {
position: static; } }
```

Текст файла src/styles/footer.css

```
.footer { background: #14261C; color: #C9D2CB; margin-top: 0; font-size: 14px; }
.footer-grid { display: grid; grid-template-columns: 2fr 1fr 1fr 1fr; gap: 40px;
padding-top: 56px; padding-bottom: 40px; }
.footer-grid > div > span.logo { display: inline-flex; }
.footer-about { margin-top: 16px; max-width: 340px; }
.footer-col { display: flex; flex-direction: column; gap: 8px; }
.footer-col h4 { color: var(--gold); font-family: var(--sans); font-size: 13px; text-
transform: uppercase; letter-spacing: .12em; margin-bottom: 8px; }
.footer-col a { color: #E6EBE7; }
.footer-col a:hover { color: var(--gold); }
.footer-bottom { border-top: 1px solid rgba(244, 239, 227, .15); padding-top: 18px;
padding-bottom: 22px; font-size: 13px; color: #9FAEA3; }
@media (max-width: 900px) { .footer-grid { grid-template-columns: 1fr 1fr; } }
```

Текст файла src/styles/header.css

```
.header { position: sticky; top: 0; z-index: 50; background: var(--paper); border-
bottom: 1px solid var(--line); }
.topbar { background: var(--green); color: var(--cream); font-size: 13px; }
.topbar-inner { display: flex; justify-content: space-between; padding-top: 7px;
padding-bottom: 7px; }
.topbar a { color: var(--gold); font-weight: 600; }
.header-inner { display: flex; align-items: center; gap: 36px; height: 76px; }
.logo { display: inline-flex; align-items: center; gap: 10px; }
.logo-text { font-family: var(--serif); font-weight: 700; font-size: 24px; letter-
spacing: .01em; white-space: nowrap; }
.logo-text span { color: var(--gold-2); margin-left: 4px; }
.nav { display: flex; gap: 28px; margin-right: auto; }
.nav a { color: var(--ink); font-weight: 500; font-size: 15px; padding: 26px 0; border-
bottom: 3px solid transparent; }
.nav a:hover { color: var(--green-2); }
.nav a.active { color: var(--green); border-bottom-color: var(--gold); }
.header-actions { display: flex; align-items: center; gap: 12px; }
.burger { display: none; background: none; border: 0; padding: 8px; cursor: pointer;
flex-direction: column; gap: 5px; }
.burger span { width: 24px; height: 2px; background: var(--green); display: block; }
@media (max-width: 900px) {
  .topbar span { display: none; }
  .burger { display: flex; }
  .nav { display: none; position: absolute; top: 100%; left: 0; right: 0; background:
var(--paper); flex-direction: column; gap: 0; padding: 8px 24px 16px; border-bottom:
1px solid var(--line); }
  .nav.nav-open { display: flex; }
  .nav a { padding: 12px 0; border: 0; }
  .header-inner { position: relative; justify-content: space-between; }
}
```

Текст файла src/styles/home.css

```
.hero { background: var(--cream); padding: 64px 0 72px; }
.hero-grid { display: grid; grid-template-columns: 1.05fr 1fr; gap: 56px; align-items:
center; }
.hero h1 { font-size: 52px; }
.hero-btns { display: flex; gap: 14px; margin-top: 28px; flex-wrap: wrap; }
.hero-img { min-height: 440px; background: var(--green-3) center/cover; border-radius:
var(--radius); box-shadow: 18px 18px 0 var(--gold); }
.stats { background: var(--green); color: var(--cream); }
.stats-grid { display: grid; grid-template-columns: repeat(4, 1fr); }
.stats-grid div { padding: 30px 24px; border-left: 1px solid rgba(244, 239, 227, .15);
}
.stats-grid div:first-child { border-left: 0; padding-left: 0; }
.stats-grid strong { display: block; font-family: var(--serif); font-size: 38px; color:
var(--gold); line-height: 1.1; }
.stats-grid span { font-size: 14px; color: #C9D2CB; }
.crops-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 26px; }
.crop-card { background: #fff; border: 1px solid var(--line); border-radius: var(--
radius); overflow: hidden; display: flex; flex-direction: column; transition: .2s; }
.crop-card:hover { box-shadow: 0 10px 28px rgba(31, 58, 43, .12); transform:
translateY(-2px); }
.crop-card-img { position: relative; display: block; aspect-ratio: 16 / 10; overflow:
hidden; background: var(--cream); }
.crop-card-img img { width: 100%; height: 100%; object-fit: cover; }
.crop-card-img .chip { position: absolute; top: 12px; left: 12px; background: var(--
gold); }
.crop-card-body { padding: 20px 22px 22px; display: flex; flex-direction: column; flex:
1; }
.crop-card-body h3 { margin-bottom: 2px; }
.crop-card-body h3 a { color: var(--green); }
.crop-card-body > p { font-size: 14px; margin-bottom: 14px; }
.crop-card-facts { margin: 0 0 18px; border-top: 1px dashed var(--line); flex: 1; }
.crop-card-facts div { display: flex; justify-content: space-between; padding: 7px 0;
border-bottom: 1px dashed var(--line); font-size: 14px; }
.crop-card-facts dt { color: var(--muted); }
.crop-card-facts dd { margin: 0; font-weight: 600; }
.crop-card-facts dd.ok { color: var(--ok); }
.crop-card-facts dd.no { color: var(--red); }
.farms-strip { display: grid; grid-template-columns: repeat(3, 1fr); gap: 18px; }
.farm-mini { height: 220px; border-radius: var(--radius); background: var(--green-3)
center/cover; position: relative; overflow: hidden; display: flex; align-items: flex-
end; }
.farm-mini > div { position: relative; padding: 18px 20px; width: 100%; background:
linear-gradient(transparent, rgba(20, 36, 26, .88)); color: var(--cream); }
.farm-mini h3 { color: #fff; margin: 2px 0; font-size: 19px; }
.farm-mini span { font-size: 12px; text-transform: uppercase; letter-spacing: .1em;
color: var(--gold); }
.farm-mini small { font-size: 13px; }
.two-col { display: grid; grid-template-columns: 1fr 1fr; gap: 64px; }
.features { display: grid; grid-template-columns: 1fr 1fr; gap: 22px; }
.feature { border-top: 3px solid var(--gold); padding-top: 14px; }
.feature h3 { font-size: 18px; margin-bottom: 6px; }
.feature p { font-size: 15px; color: #4A544D; margin: 0; }
.steps { list-style: none; padding: 0; margin: 0; }
.steps li { display: flex; gap: 18px; padding: 14px 0; border-bottom: 1px solid var(--
line); }
.steps li > span { flex: 0 0 42px; height: 42px; border-radius: 50%; background: var(--
green); color: var(--gold); font: 700 18px/42px var(--serif); text-align: center; }
.steps h3 { font-size: 18px; margin-bottom: 4px; }
.steps p { font-size: 15px; color: #4A544D; margin: 0; }
@media (max-width: 900px) {
  .hero-grid, .two-col { grid-template-columns: 1fr; gap: 32px; }
  .hero h1 { font-size: 36px; }
  .hero-img { min-height: 260px; box-shadow: 10px 10px 0 var(--gold); }
  .stats-grid, .crops-grid, .farms-strip, .features { grid-template-columns: 1fr 1fr; }
}
```

Текст файла src/styles/product.css

```
.product { padding: 28px 24px 72px; }
.back { display: inline-block; margin-bottom: 20px; font-weight: 500; }
.product-top { display: grid; grid-template-columns: 1fr 1fr; gap: 48px; margin-bottom:
48px; align-items: center; }
.product-img img { width: 100%; aspect-ratio: 4 / 3; object-fit: cover; border-radius:
var(--radius); }
.product-info h1 { margin: 12px 0 4px; }
.grade { color: var(--gold-2); font-weight: 600; }
.price-row { display: flex; gap: 0; margin-top: 24px; border: 1px solid var(--line);
border-radius: var(--radius); background: #fff; }
.price-row div { flex: 1; padding: 16px 20px; }
.price-row div + div { border-left: 1px solid var(--line); }
.price-row small { display: block; color: var(--muted); font-size: 13px; }
.price-row strong { font-family: var(--serif); font-size: 28px; color: var(--green); }
.product-grid { display: grid; grid-template-columns: 1fr 360px; gap: 40px; align-
items: start; }
.product-grid h2 { font-size: 26px; }
.product-grid tbody tr { cursor: pointer; }
.product-grid tbody tr.active td { background: #FFF6DF; }
.order-box { background: #fff; border: 1px solid var(--line); border-top: 5px solid
var(--gold); border-radius: var(--radius); padding: 24px; position: sticky; top: 130px;
}
.order-box form { display: flex; flex-direction: column; gap: 14px; }
.order-total { display: flex; justify-content: space-between; align-items: baseline;
padding: 12px 0; border-top: 1px dashed var(--line); }
.order-total strong { font-family: var(--serif); font-size: 26px; color: var(--green);
}
@media (max-width: 900px) { .product-top, .product-grid { grid-template-columns: 1fr; }
.order-box { position: static; } }
```

Текст файла db/agro_db.sql

```
SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;
DROP DATABASE IF EXISTS `agro_db`;
CREATE DATABASE `agro_db` DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE `agro_db`;


CREATE TABLE `users` (
  `id_user` INT AUTO_INCREMENT PRIMARY KEY,
  `last_name` VARCHAR(60) NOT NULL,
  `first_name` VARCHAR(60) NOT NULL,
  `company` VARCHAR(150) NULL,
  `inn` VARCHAR(12) NULL,
  `email` VARCHAR(120) NOT NULL UNIQUE,
  `phone` VARCHAR(20) NULL,
  `password_hash` VARCHAR(255) NOT NULL,
  `role` ENUM('client','manager','admin') NOT NULL DEFAULT 'client',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;


CREATE TABLE `farms` (
  `id_farm` INT AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(120) NOT NULL UNIQUE,
  `region` VARCHAR(80) NOT NULL,
  `district` VARCHAR(80) NOT NULL,
  `area_ha` DECIMAL(10,1) NOT NULL,
  `specialization` VARCHAR(150) NOT NULL,
  `image` VARCHAR(255) NULL,
  CONSTRAINT `chk_farm_area` CHECK (`area_ha` > 0)
) ENGINE=InnoDB;


CREATE TABLE `fields` (
```

```
  `id_field` INT AUTO_INCREMENT PRIMARY KEY,
  `id_farm` INT NOT NULL,
  `cadastral_number` VARCHAR(30) NOT NULL UNIQUE,
  `area_ha` DECIMAL(8,1) NOT NULL,
  `soil_type` ENUM('Чернозём','Каштановая','Лугово-чернозёмная','Серая лесная') NOT
NULL,
  CONSTRAINT `fk_field_farm` FOREIGN KEY (`id_farm`) REFERENCES `farms`(`id_farm`) ON
UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT `chk_field_area` CHECK (`area_ha` > 0)
) ENGINE=InnoDB;


CREATE TABLE `crops` (
  `id_crop` INT AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(100) NOT NULL UNIQUE,
  `category` ENUM('Зерновые','Масличные','Бобовые','Технические') NOT NULL,
  `grade` VARCHAR(60) NULL,
  `description` TEXT NULL,
  `price_per_ton` DECIMAL(10,2) NOT NULL,
  `image` VARCHAR(255) NULL,
  `is_active` TINYINT(1) NOT NULL DEFAULT 1,
  CONSTRAINT `chk_crop_price` CHECK (`price_per_ton` > 0)
) ENGINE=InnoDB;


CREATE TABLE `sowings` (
  `id_sowing` INT AUTO_INCREMENT PRIMARY KEY,
  `id_field` INT NOT NULL,
  `id_crop` INT NOT NULL,
  `season` SMALLINT NOT NULL,
  `sown_date` DATE NOT NULL,
  `area_ha` DECIMAL(8,1) NOT NULL,
  `planned_yield` DECIMAL(5,1) NOT NULL,
  CONSTRAINT `fk_sow_field` FOREIGN KEY (`id_field`) REFERENCES `fields`(`id_field`) ON
UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT `fk_sow_crop` FOREIGN KEY (`id_crop`) REFERENCES `crops`(`id_crop`) ON
UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT `uq_sow` UNIQUE (`id_field`, `season`),
  CONSTRAINT `chk_sow` CHECK (`area_ha` > 0 AND `planned_yield` > 0)
) ENGINE=InnoDB;


CREATE TABLE `harvests` (
  `id_harvest` INT AUTO_INCREMENT PRIMARY KEY,
  `id_sowing` INT NOT NULL,
  `harvest_date` DATE NOT NULL,
  `amount_t` DECIMAL(10,1) NOT NULL,
  `moisture` DECIMAL(4,1) NOT NULL,
  `quality_class` VARCHAR(30) NULL,
  CONSTRAINT `fk_harv_sow` FOREIGN KEY (`id_sowing`) REFERENCES `sowings`(`id_sowing`)
ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT `chk_harv` CHECK (`amount_t` > 0 AND `moisture` BETWEEN 5 AND 40)
) ENGINE=InnoDB;


CREATE TABLE `warehouses` (
  `id_warehouse` INT AUTO_INCREMENT PRIMARY KEY,
  `id_farm` INT NOT NULL,
  `name` VARCHAR(120) NOT NULL,
  `type` ENUM('Элеватор','Склад напольного хранения','Силосный склад') NOT NULL,
  `address` VARCHAR(200) NOT NULL,
  `capacity_t` DECIMAL(10,1) NOT NULL,
  CONSTRAINT `fk_wh_farm` FOREIGN KEY (`id_farm`) REFERENCES `farms`(`id_farm`) ON
UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT `chk_wh` CHECK (`capacity_t` > 0)
) ENGINE=InnoDB;


CREATE TABLE `stock` (
  `id_stock` INT AUTO_INCREMENT PRIMARY KEY,
```

```
  `id_warehouse` INT NOT NULL,
  `id_crop` INT NOT NULL,
  `quantity_t` DECIMAL(10,1) NOT NULL DEFAULT 0,
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT `fk_stock_wh` FOREIGN KEY (`id_warehouse`) REFERENCES
`warehouses`(`id_warehouse`) ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT `fk_stock_crop` FOREIGN KEY (`id_crop`) REFERENCES `crops`(`id_crop`) ON
UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT `uq_stock` UNIQUE (`id_warehouse`, `id_crop`),
  CONSTRAINT `chk_stock` CHECK (`quantity_t` >= 0)
) ENGINE=InnoDB;


CREATE TABLE `orders` (
  `id_order` INT AUTO_INCREMENT PRIMARY KEY,
  `id_user` INT NOT NULL,
  `id_crop` INT NOT NULL,
  `id_warehouse` INT NOT NULL,
  `quantity_t` DECIMAL(10,1) NOT NULL,
  `price_per_ton` DECIMAL(10,2) NOT NULL,
  `total_price` DECIMAL(14,2) NOT NULL,
  `delivery` ENUM('Самовывоз','Автодоставка','Железная дорога') NOT NULL DEFAULT
'Самовывоз',
  `address` VARCHAR(200) NULL,
  `status` ENUM('new','confirmed','shipped','completed','cancelled') NOT NULL DEFAULT
'new',
  `comment` VARCHAR(500) NULL,
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT `fk_ord_user` FOREIGN KEY (`id_user`) REFERENCES `users`(`id_user`) ON
UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT `fk_ord_crop` FOREIGN KEY (`id_crop`) REFERENCES `crops`(`id_crop`) ON
UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT `fk_ord_wh` FOREIGN KEY (`id_warehouse`) REFERENCES
`warehouses`(`id_warehouse`) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT `chk_ord` CHECK (`quantity_t` >= 5 AND `quantity_t` <= 5000)
) ENGINE=InnoDB;


CREATE TABLE `requests` (
  `id_request` INT AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(100) NOT NULL,
  `company` VARCHAR(150) NULL,
  `phone` VARCHAR(20) NOT NULL,
  `email` VARCHAR(120) NULL,
  `topic` ENUM('Закупка продукции','Сотрудничество','Аренда земли','Работа в
компании','Другое') NOT NULL DEFAULT 'Закупка продукции',
  `message` VARCHAR(1000) NULL,
  `status` ENUM('new','in_work','closed') NOT NULL DEFAULT 'new',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;



INSERT INTO `users` (`last_name`, `first_name`, `company`, `inn`, `email`, `phone`,
`password_hash`, `role`, `created_at`) VALUES
('Григорьева', 'Анна', 'АО «Колос-Юг»', '2310123456', 'admin@kolos-yug.ru', '+7 (861)
200-10-00', '$2y$10$BlxL/UJSBae2InbA0sMdy.4Tc7PyQD6paVHyvRroZ5BBZI6hWJG7q', 'admin',
'2026-01-12 09:00:00'),
('Савченко', 'Игорь', 'АО «Колос-Юг»', '2310123456', 'manager@kolos-yug.ru', '+7 (861)
200-10-01', '$2y$10$h.USpYOEy4ONI1oLKc3toOe6eXvmkn1Ao6/iK.3tj4wwYviK.129m', 'manager',
'2026-01-15 10:00:00'),
('Петренко', 'Олег', 'ООО «Мелькомбинат Юг»', '6164123450', 'petrenko@melkomb-yug.ru',
'+7 (863) 251-44-10', '$2y$10$PVMQgbz28FLaVAfqlwwwAOBydGx22SH9Q9793XnSSmhjCi8dRHwqa',
'client', '2026-02-03 11:20:00'),
('Абрамова', 'Светлана', 'АО «Хлебозавод № 3»', '7701234561', 'abramova@hleb3.ru', '+7
(495) 744-21-30', '$2y$10$PVMQgbz28FLaVAfqlwwwAOBydGx22SH9Q9793XnSSmhjCi8dRHwqa',
'client', '2026-03-11 15:40:00'),
```

```
('Кравцов', 'Денис', 'ООО «МаслоПром»', '3664098712', 'kravtsov@masloprom.ru', '+7
(473) 290-55-17', '$2y$10$PVMQgbz28FLaVAfqlwwwAOBydGx22SH9Q9793XnSSmhjCi8dRHwqa',
'client', '2026-04-02 09:05:00'),
('Бондаренко', 'Виктор', 'ООО «Комбикорм-Дон»', '6150087654', 'bondarenko@kombikorm-
don.ru', '+7 (863) 245-02-88',
'$2y$10$PVMQgbz28FLaVAfqlwwwAOBydGx22SH9Q9793XnSSmhjCi8dRHwqa', 'client', '2026-05-19
13:30:00'),
('Мельник', 'Юлия', 'ООО «АгроТрейд Экспорт»', '2308765432', 'melnik@agrotrade-
export.ru', '+7 (861) 279-61-40',
'$2y$10$PVMQgbz28FLaVAfqlwwwAOBydGx22SH9Q9793XnSSmhjCi8dRHwqa', 'client', '2026-06-25
16:10:00'),
('Тарасов', 'Павел', 'ООО «Сахар Черноземья»', '3123456780', 'tarasov@sahar-ch.ru', '+7
(472) 233-18-90', '$2y$10$PVMQgbz28FLaVAfqlwwwAOBydGx22SH9Q9793XnSSmhjCi8dRHwqa',
'client', '2026-07-08 10:45:00');


INSERT INTO `farms` (`name`, `region`, `district`, `area_ha`, `specialization`,
`image`) VALUES
('Колос-Юг Кубань', 'Краснодарский край', 'Тимашевский район', 18400, 'Озимая пшеница,
кукуруза, подсолнечник', 'farm_kuban.jpg'),
('Колос-Юг Дон', 'Ростовская область', 'Сальский район', 22600, 'Озимая пшеница,
ячмень, подсолнечник', 'farm_don.jpg'),
('Колос-Юг Ставрополье', 'Ставропольский край', 'Ипатовский район', 15900, 'Озимая
пшеница, горох, рапс', 'farm_stavropol.jpg'),
('Колос-Юг Черноземье', 'Воронежская область', 'Россошанский район', 12300, 'Соя,
кукуруза, сахарная свёкла', 'farm_voronezh.jpg'),
('Колос-Юг Белогорье', 'Белгородская область', 'Алексеевский район', 9800, 'Соя,
ячмень, сахарная свёкла', 'farm_belgorod.jpg'),
('Колос-Юг Курск', 'Курская область', 'Обоянский район', 8700, 'Кукуруза, подсолнечник,
рапс', 'farm_kursk.jpg');


INSERT INTO `fields` (`id_farm`, `cadastral_number`, `area_ha`, `soil_type`) VALUES
(1, '23:31:2885739:792', 393.1, 'Чернозём'),
(1, '23:31:1725920:152', 800.3, 'Чернозём'),
(1, '23:31:9979369:247', 1396.8, 'Чернозём'),
(1, '23:31:3341959:160', 1141.3, 'Чернозём'),
(2, '61:34:1555588:688', 1430.4, 'Каштановая'),
(2, '61:34:4878375:558', 444.9, 'Каштановая'),
(2, '61:34:9825979:432', 1163.6, 'Каштановая'),
(2, '61:34:2562149:130', 599.3, 'Каштановая'),
(3, '26:02:7760308:359', 401.8, 'Чернозём'),
(3, '26:02:6541672:411', 592.4, 'Чернозём'),
(3, '26:02:5136646:225', 770.4, 'Чернозём'),
(3, '26:02:7984988:608', 1154.7, 'Чернозём'),
(4, '36:27:2007521:986', 1089.1, 'Чернозём'),
(4, '36:27:4018057:374', 435.2, 'Чернозём'),
(4, '36:27:3476737:866', 524.2, 'Чернозём'),
(4, '36:27:7315146:41', 1231.8, 'Чернозём'),
(5, '31:01:9912616:899', 472.1, 'Серая лесная'),
(5, '31:01:4663456:222', 849.4, 'Серая лесная'),
(5, '31:01:7165153:208', 1232.4, 'Серая лесная'),
(5, '31:01:4677284:503', 733.8, 'Серая лесная'),
(6, '46:17:6637313:310', 424.3, 'Лугово-чернозёмная'),
(6, '46:17:7646563:503', 737.7, 'Лугово-чернозёмная'),
(6, '46:17:6307918:994', 1072.0, 'Лугово-чернозёмная'),
(6, '46:17:2598981:818', 608.1, 'Лугово-чернозёмная');


INSERT INTO `crops` (`name`, `category`, `grade`, `description`, `price_per_ton`,
`image`) VALUES
('Пшеница озимая', 'Зерновые', '3 класс, ГОСТ 9353-2016', 'Продовольственная пшеница с
содержанием протеина от 12,5% и клейковины от 23%. Подходит для мукомольных
предприятий.', 17500, 'wheat.jpg'),
('Пшеница фуражная', 'Зерновые', '5 класс', 'Фуражное зерно для комбикормовых заводов и
животноводческих комплексов.', 14800, 'wheat_feed.jpg'),
('Ячмень пивоваренный', 'Зерновые', '1 класс, ГОСТ 5060-86', 'Двурядный ячмень с белком
до 12% и способностью прорастания от 95%.', 16200, 'barley.jpg'),
```

```
('Кукуруза на зерно', 'Зерновые', '2 класс, ГОСТ 13634-90', 'Кукуруза с влажностью до
14% для крахмало-паточной и комбикормовой промышленности.', 15600, 'corn.jpg'),
('Подсолнечник', 'Масличные', 'Высокоолеиновый', 'Семена подсолнечника масличностью от
48% для маслоэкстракционных заводов.', 38500, 'sunflower.jpg'),
('Рапс озимый', 'Масличные', '00-тип', 'Безэруковый низкоглюкозинолатный рапс для
производства масла и шрота.', 42000, 'rapeseed.jpg'),
('Соя', 'Бобовые', 'ГОСТ 17109-88', 'Соя с содержанием белка от 36% для переработки в
шрот и масло.', 40500, 'soy.jpg'),
('Горох', 'Бобовые', 'Продовольственный', 'Горох жёлтый лущёный для пищевой
промышленности и экспорта.', 22500, 'peas.jpg'),
('Сахарная свёкла', 'Технические', 'Дигестия от 16%', 'Сахарная свёкла для сахарных
заводов Черноземья, поставка в сезон переработки.', 4800, 'beet.jpg');


INSERT INTO `sowings` (`id_field`, `id_crop`, `season`, `sown_date`, `area_ha`,
`planned_yield`) VALUES
(1, 5, 2025, '2025-05-08', 393.1, 30),
(1, 4, 2026, '2026-05-05', 393.1, 75),
(2, 1, 2025, '2024-09-26', 800.3, 60),
(2, 5, 2026, '2026-04-06', 800.3, 30),
(3, 1, 2025, '2024-09-13', 1396.8, 60),
(3, 1, 2026, '2025-09-11', 1396.8, 60),
(4, 4, 2025, '2025-04-18', 1141.3, 75),
(4, 1, 2026, '2025-10-16', 1141.3, 60),
(5, 5, 2025, '2025-04-18', 1430.4, 30),
(5, 3, 2026, '2026-05-27', 1430.4, 50),
(6, 2, 2025, '2024-10-16', 444.9, 55),
(6, 5, 2026, '2026-05-17', 444.9, 30),
(7, 1, 2025, '2024-09-24', 1163.6, 60),
(7, 2, 2026, '2025-10-27', 1163.6, 55),
(8, 3, 2025, '2025-05-10', 599.3, 50),
(8, 1, 2026, '2025-10-27', 599.3, 60),
(9, 6, 2025, '2024-10-28', 401.8, 32),
(9, 8, 2026, '2026-04-04', 401.8, 35),
(10, 1, 2025, '2024-09-12', 592.4, 60),
(10, 6, 2026, '2025-10-22', 592.4, 32),
(11, 1, 2025, '2024-09-06', 770.4, 60),
(11, 1, 2026, '2025-10-22', 770.4, 60),
(12, 8, 2025, '2025-04-15', 1154.7, 35),
(12, 1, 2026, '2025-09-12', 1154.7, 60),
(13, 9, 2025, '2025-05-06', 1089.1, 480),
(13, 4, 2026, '2026-05-18', 1089.1, 75),
(14, 1, 2025, '2024-09-20', 435.2, 60),
(14, 9, 2026, '2026-04-03', 435.2, 480),
(15, 7, 2025, '2025-05-15', 524.2, 25),
(15, 1, 2026, '2025-09-23', 524.2, 60),
(16, 4, 2025, '2025-04-10', 1231.8, 75),
(16, 7, 2026, '2026-05-06', 1231.8, 25),
(17, 9, 2025, '2025-04-03', 472.1, 480),
(17, 3, 2026, '2026-05-24', 472.1, 50),
(18, 1, 2025, '2024-10-17', 849.4, 60),
(18, 9, 2026, '2026-04-27', 849.4, 480),
(19, 7, 2025, '2025-05-04', 1232.4, 25),
(19, 1, 2026, '2025-09-18', 1232.4, 60),
(20, 3, 2025, '2025-04-28', 733.8, 50),
(20, 7, 2026, '2026-05-27', 733.8, 25),
(21, 6, 2025, '2024-09-06', 424.3, 32),
(21, 5, 2026, '2026-05-10', 424.3, 30),
(22, 2, 2025, '2024-10-20', 737.7, 55),
(22, 6, 2026, '2025-09-26', 737.7, 32),
(23, 4, 2025, '2025-04-17', 1072.0, 75),
(23, 2, 2026, '2025-10-24', 1072.0, 55),
(24, 5, 2025, '2025-04-24', 608.1, 30),
(24, 4, 2026, '2026-05-27', 608.1, 75);


INSERT INTO `harvests` (`id_sowing`, `harvest_date`, `amount_t`, `moisture`,
`quality_class`) VALUES
```

```
(1, '2025-09-22', 1196.6, 7.6, 'Масличность 49%'),
(2, '2026-10-05', 3236.8, 14.2, '2 класс'),
(3, '2025-07-17', 4416.4, 12.2, '3 класс'),
(4, '2026-09-15', 2315.7, 8.0, 'Масличность 49%'),
(5, '2025-07-08', 9088.7, 12.7, '3 класс'),
(6, '2026-07-15', 7420.8, 13.3, '3 класс'),
(7, '2025-10-06', 8384.1, 14.5, '2 класс'),
(8, '2026-07-24', 6692.2, 12.6, '3 класс'),
(9, '2025-09-09', 4562.6, 7.7, 'Масличность 49%'),
(10, '2026-07-05', 6517.8, 13.2, '1 класс'),
(11, '2025-07-12', 2063.1, 12.8, '5 класс'),
(12, '2026-09-06', 1170.5, 7.6, 'Масличность 49%'),
(13, '2025-07-11', 7406.8, 13.2, '3 класс'),
(14, '2026-07-10', 6369.5, 13.1, '5 класс'),
(15, '2025-07-21', 2680.4, 13.0, '1 класс'),
(16, '2026-07-19', 3348.7, 12.3, '3 класс'),
(17, '2025-07-11', 1419.4, 8.3, 'Масличность 44%'),
(18, '2026-07-26', 1228.1, 13.4, '1 класс'),
(19, '2025-07-17', 3864.0, 12.0, '3 класс'),
(20, '2026-07-23', 1721.2, 8.4, 'Масличность 44%'),
(21, '2025-07-20', 4678.4, 13.1, '3 класс'),
(22, '2026-07-11', 4445.7, 12.8, '3 класс'),
(23, '2025-07-06', 4320.1, 13.3, '1 класс'),
(24, '2026-07-11', 6444.3, 12.2, '3 класс'),
(25, '2025-10-11', 44878.0, 21.6, 'Дигестия 17,2%'),
(26, '2026-10-09', 8366.8, 13.9, '2 класс'),
(27, '2025-07-09', 2410.7, 12.5, '3 класс'),
(29, '2025-09-13', 1378.8, 11.7, 'Белок 37%'),
(30, '2026-07-14', 2997.3, 12.9, '3 класс'),
(31, '2025-10-24', 8128.8, 14.2, '2 класс'),
(33, '2025-10-16', 24785.3, 22.0, 'Дигестия 17,2%'),
(34, '2026-07-20', 2326.1, 13.4, '1 класс'),
(35, '2025-07-17', 5247.3, 12.6, '3 класс'),
(37, '2025-09-23', 2797.1, 12.3, 'Белок 37%'),
(38, '2026-07-26', 8100.5, 12.1, '3 класс'),
(39, '2025-07-22', 3145.3, 12.4, '1 класс'),
(41, '2025-07-14', 1259.4, 7.3, 'Масличность 44%'),
(42, '2026-09-19', 1263.8, 6.8, 'Масличность 49%'),
(43, '2025-07-19', 3716.9, 13.4, '5 класс'),
(44, '2026-07-28', 2412.5, 7.6, 'Масличность 44%'),
(45, '2025-10-16', 6881.7, 14.1, '2 класс'),
(46, '2026-07-17', 5286.4, 12.6, '5 класс'),
(47, '2025-09-19', 1552.3, 7.9, 'Масличность 49%'),
(48, '2026-10-17', 4993.1, 13.9, '2 класс');

INSERT INTO `warehouses` (`id_farm`, `name`, `type`, `address`, `capacity_t`) VALUES
(1, 'Тимашевский элеватор', 'Элеватор', 'Краснодарский край, г. Тимашевск, ул.
Элеваторная, 1', 60000),
(1, 'Склад ОП «Медведовская»', 'Склад напольного хранения', 'Краснодарский край, ст.
Медведовская, ул. Полевая, 12', 18000),
(2, 'Сальский элеватор', 'Элеватор', 'Ростовская область, г. Сальск, ул. Заводская, 5',
75000),
(3, 'Ипатовский элеватор', 'Элеватор', 'Ставропольский край, г. Ипатово, ул. Гагарина,
140', 50000),
(4, 'Россошанский силосный склад', 'Силосный склад', 'Воронежская область, г. Россошь,
ул. Промышленная, 9', 30000),
(5, 'Склад «Алексеевка»', 'Склад напольного хранения', 'Белгородская область, г.
Алексеевка, ул. Мостовая, 77', 20000),
(6, 'Обоянский элеватор', 'Элеватор', 'Курская область, г. Обоянь, ул. Луначарского,
3', 35000);

INSERT INTO `stock` (`id_warehouse`, `id_crop`, `quantity_t`) VALUES
(1, 1, 4336.1),
(1, 4, 5502.3),
(1, 5, 3813.3),
(2, 1, 660.0),
```

```
(2, 5, 1345.7),
(3, 1, 4952.6),
(3, 2, 4949.8),
(3, 3, 1781.9),
(3, 5, 2869.6),
(4, 1, 2221.5),
(4, 6, 2699.3),
(4, 8, 5327.8),
(5, 1, 1215.8),
(5, 4, 1032.5),
(5, 7, 963.3),
(6, 1, 832.4),
(6, 3, 1092.3),
(6, 7, 721.6),
(7, 2, 952.8),
(7, 4, 1031.8),
(7, 5, 1098.9),
(7, 6, 2380.7);


INSERT INTO `orders` (`id_user`, `id_crop`, `id_warehouse`, `quantity_t`,
`price_per_ton`, `total_price`, `delivery`, `address`, `status`, `comment`,
`created_at`) VALUES
(3, 1, 1, 600, 17500, 10500000, 'Самовывоз', NULL, 'completed', NULL, '2026-08-04
10:30:00'),
(4, 1, 3, 350, 17500, 6125000, 'Автодоставка', 'г. Москва, ул. Складочная, 1',
'completed', NULL, '2026-09-22 16:45:00'),
(5, 5, 1, 900, 38500, 34650000, 'Железная дорога', 'ст. Новороссийск-Порт, тупик 4',
'completed', NULL, '2026-08-03 17:00:00'),
(6, 2, 3, 1200, 14800, 17760000, 'Самовывоз', NULL, 'completed', NULL, '2026-09-26
17:45:00'),
(7, 1, 1, 2500, 17500, 43750000, 'Автодоставка', 'г. Москва, ул. Складочная, 1',
'shipped', NULL, '2026-10-12 11:45:00'),
(5, 6, 4, 400, 42000, 16800000, 'Железная дорога', 'ст. Новороссийск-Порт, тупик 4',
'shipped', NULL, '2026-10-05 09:30:00'),
(3, 1, 3, 800, 17500, 14000000, 'Самовывоз', NULL, 'confirmed', NULL, '2026-09-11
13:00:00'),
(6, 4, 5, 700, 15600, 10920000, 'Автодоставка', 'г. Москва, ул. Складочная, 1',
'confirmed', NULL, '2026-09-04 18:15:00'),
(7, 4, 1, 1500, 15600, 23400000, 'Железная дорога', 'ст. Новороссийск-Порт, тупик 4',
'confirmed', NULL, '2026-08-09 11:30:00'),
(8, 7, 5, 300, 40500, 12150000, 'Самовывоз', NULL, 'new', NULL, '2026-10-20 16:30:00'),
(4, 3, 3, 150, 16200, 2430000, 'Автодоставка', 'г. Москва, ул. Складочная, 1', 'new',
NULL, '2026-09-24 10:15:00'),
(5, 5, 7, 650, 38500, 25025000, 'Железная дорога', 'ст. Новороссийск-Порт, тупик 4',
'new', NULL, '2026-08-23 11:45:00'),
(6, 3, 3, 500, 16200, 8100000, 'Самовывоз', NULL, 'cancelled', NULL, '2026-08-14
16:15:00'),
(3, 1, 4, 450, 17500, 7875000, 'Автодоставка', 'г. Москва, ул. Складочная, 1', 'new',
NULL, '2026-08-15 17:15:00'),
(7, 8, 4, 600, 22500, 13500000, 'Железная дорога', 'ст. Новороссийск-Порт, тупик 4',
'completed', NULL, '2026-09-26 15:45:00'),
(8, 7, 6, 250, 40500, 10125000, 'Самовывоз', NULL, 'confirmed', NULL, '2026-08-27
16:30:00');


INSERT INTO `requests` (`name`, `company`, `phone`, `email`, `topic`, `message`,
`status`, `created_at`) VALUES
('Алексей', 'ООО «ЗерноЭкспорт»', '+7 (918) 455-20-11', 'a.ivlev@zernoexport.ru',
'Закупка продукции', 'Интересует пшеница 4 класса, 5 000 т на условиях FOB
Новороссийск, поставка в ноябре.', 'new', '2026-10-18 10:20:00'),
('Наталья', 'КФХ Степанова', '+7 (928) 300-14-75', NULL, 'Аренда земли', 'Предлагаем в
аренду 420 га пашни в Сальском районе.', 'new', '2026-10-19 14:05:00'),
('Роман', 'ООО «Агрохимпром»', '+7 (495) 921-66-02', 'sales@agrohimprom.ru',
'Сотрудничество', 'Готовы предложить средства защиты растений на сезон 2027 года.',
'in_work', '2026-10-10 09:40:00'),
('Ирина', NULL, '+7 (961) 520-33-48', 'irina.k@mail.ru', 'Работа в компании', 'Ищу
вакансию агронома в Ставропольском крае.', 'closed', '2026-09-22 16:15:00'),
```

```
('Михаил', 'ИП Гусейнов', '+7 (909) 444-12-87', 'guseynov.m@yandex.ru', 'Закупка
продукции', 'Нужен горох 60 т для фасовки, самовывоз.', 'new', '2026-10-20 11:30:00');



CREATE OR REPLACE VIEW `view1_products_catalog` AS
SELECT c.id_crop, c.name, c.category, c.grade, c.price_per_ton,
       COALESCE(SUM(s.quantity_t), 0) AS stock_t, COUNT(DISTINCT CASE WHEN s.quantity_t
> 0 THEN s.id_warehouse END) AS warehouses
FROM crops c LEFT JOIN stock s ON s.id_crop = c.id_crop
WHERE c.is_active = 1
GROUP BY c.id_crop, c.name, c.category, c.grade, c.price_per_ton;

CREATE OR REPLACE VIEW `view2_warehouse_load` AS
SELECT w.id_warehouse, w.name AS warehouse, f.name AS farm, w.capacity_t,
       COALESCE(SUM(s.quantity_t), 0) AS stored_t,
       ROUND(COALESCE(SUM(s.quantity_t), 0) * 100 / w.capacity_t, 1) AS load_percent
FROM warehouses w JOIN farms f ON f.id_farm = w.id_farm
LEFT JOIN stock s ON s.id_warehouse = w.id_warehouse
GROUP BY w.id_warehouse, w.name, f.name, w.capacity_t;

CREATE OR REPLACE VIEW `view3_orders_full` AS
SELECT o.id_order, u.company, CONCAT(u.last_name, ' ', u.first_name) AS contact, c.name
AS crop,
       w.name AS warehouse, o.quantity_t, o.total_price, o.delivery, o.status,
o.created_at
FROM orders o
JOIN users u ON u.id_user = o.id_user
JOIN crops c ON c.id_crop = o.id_crop
JOIN warehouses w ON w.id_warehouse = o.id_warehouse;

CREATE OR REPLACE VIEW `view4_revenue_by_crop` AS
SELECT c.name AS crop, COUNT(o.id_order) AS orders_count, COALESCE(SUM(o.quantity_t),
0) AS sold_t,
       COALESCE(SUM(o.total_price), 0) AS revenue
FROM crops c
LEFT JOIN orders o ON o.id_crop = c.id_crop AND o.status IN ('confirmed', 'shipped',
'completed')
GROUP BY c.name;

CREATE OR REPLACE VIEW `view5_field_yield` AS
SELECT fl.cadastral_number, f.name AS farm, c.name AS crop, s.season, s.area_ha,
       h.amount_t, ROUND(h.amount_t * 10 / s.area_ha, 1) AS yield_c_ha
FROM harvests h
JOIN sowings s ON s.id_sowing = h.id_sowing
JOIN fields fl ON fl.id_field = s.id_field
JOIN farms f ON f.id_farm = fl.id_farm
JOIN crops c ON c.id_crop = s.id_crop;

CREATE OR REPLACE VIEW `view6_farm_areas` AS
SELECT f.name AS farm, f.region, COUNT(DISTINCT fl.id_field) AS fields_count,
       COALESCE(SUM(fl.area_ha), 0) AS fields_area_ha, f.area_ha AS total_area_ha
FROM farms f LEFT JOIN fields fl ON fl.id_farm = f.id_farm
GROUP BY f.id_farm, f.name, f.region, f.area_ha;

CREATE OR REPLACE VIEW `view7_plan_vs_fact` AS
SELECT s.season, c.name AS crop, SUM(s.area_ha) AS area_ha,
       ROUND(SUM(s.area_ha * s.planned_yield) / 10, 1) AS plan_t,
       ROUND(COALESCE(SUM(h.amount_t), 0), 1) AS fact_t,
       ROUND(COALESCE(SUM(h.amount_t), 0) * 1000 / SUM(s.area_ha * s.planned_yield), 1)
AS completion_percent
FROM sowings s
JOIN crops c ON c.id_crop = s.id_crop
LEFT JOIN harvests h ON h.id_sowing = s.id_sowing
```

```
GROUP BY s.season, c.name;


CREATE OR REPLACE VIEW `view8_client_stats` AS
SELECT u.id_user, u.company, u.inn, COUNT(o.id_order) AS orders_count,
       COALESCE(SUM(o.quantity_t), 0) AS bought_t, COALESCE(SUM(o.total_price), 0) AS
total_sum
FROM users u
LEFT JOIN orders o ON o.id_user = u.id_user AND o.status <> 'cancelled'
WHERE u.role = 'client'
GROUP BY u.id_user, u.company, u.inn;


CREATE OR REPLACE VIEW `view9_new_requests` AS
SELECT id_request, name, company, phone, email, topic, message, created_at
FROM requests WHERE status = 'new';


CREATE OR REPLACE VIEW `view10_monthly_sales` AS
SELECT DATE_FORMAT(created_at, '%Y-%m') AS month, COUNT(*) AS orders_count,
       SUM(quantity_t) AS sold_t, SUM(total_price) AS total_sum
FROM orders WHERE status <> 'cancelled'
GROUP BY DATE_FORMAT(created_at, '%Y-%m');


SET FOREIGN_KEY_CHECKS = 1;
```